



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA
**Dipartimento di Informatica, Sistemistica e
Comunicazione**
Corso di Laurea in Informatica

Generazione di prefissi dell'Unfolding di una rete di Petri

Relatore: Luca Bernardinello

Tesi di Laurea di:
Andrea Marco Loprete
Matricola 852234

Anno Accademico 2024-2025

Indice

Introduzione	2
1 Definizioni preliminari	4
1.1 Rete di Petri	4
1.2 Branching Processes	8
1.3 Configurazioni e tagli	11
2 Algoritmi per la costruzione di prefissi finiti	12
2.1 Costruzione dell'unfolding	12
2.2 Costruzione di prefissi finiti completi	13
2.3 Un ordine adeguato per reti di Petri	16
2.4 Un ordine totale per reti 1-safe	18
2.5 Trovare possibili estensioni	19
2.6 Utilizzi dei prefissi dell'unfolding di una rete di Petri	23
3 Directed Unfolding di reti di Petri	26
3.1 Dirigere l'unfolding	27
3.2 La dimensione del prefisso finito	28
3.3 Euristiche	29
4 Implementazione di un programma per la costruzione di prefissi finiti	32
4.1 Specifiche tecniche	32
4.2 Parser di reti di Petri	35
4.3 Il modulo net.py e la rete di Petri	36
4.4 Il modulo branching_process.py e l'unfolding della rete di Petri	37
4.5 Relazioni di conflitto, causalità e concorrenza	42
4.6 Ordine adeguato ed eventi cut-off	46
4.7 Conversione in linguaggio DOT	49
4.8 Esempi di unfolding	51
Conclusioni	54

Introduzione

Le reti di Petri furono introdotte nel 1962, da Carl Adam Petri, nella sua tesi di dottorato. Si tratta di uno strumento di modellazione matematica e grafica in grado di modellare sistemi concorrenti e distribuiti, rappresentando in modo semplice e intuitivo eventi, condizioni e relazioni tra di essi, tramite un grafo orientato tra le condizioni, chiamate *posti* (places), e i possibili eventi, le *transizioni* (transitions). Ogni posto è in grado di contenere delle risorse, i *token*, che possono "scorrere" da posto a posto tramite l'occorrenza delle transizioni, rappresentando il flusso d'esecuzione tra gli stati di un processo distribuito e concorrente. Sono impiegate in numerosi ambiti: il controllo di processi di automazione industriale, gestioni dei flussi di lavoro, per simulare le differenti esecuzioni di un sistema o per la verifica delle proprietà di un sistema, come la raggiungibilità di determinati stati o la presenza di deadlock.

Esistono varie classi di reti di Petri, ognuna con differenti caratteristiche e applicazioni. Le reti oggetto di analisi in questo elaborato sono le reti posto/-transizione, dette P/T: i posti possono contenere uno o più token, possono avere una capacità massima di token e gli archi tra posti e transizioni hanno un peso, ovvero il numero di token richiesti dall'esecuzione di una transizione [1]. Una sottoclasse delle reti P/T sono le reti 1-safe, nelle quali i posti possono contenere al massimo un solo token.

Reti alternative e particolari sono le reti temporizzate, reti dove le transizioni possono implementare un ritardo di tempo per rappresentare al meglio sistemi dinamici o di gestione dei processi e possono essere reti *deterministiche*, se i tempi sono assegnati in maniera deterministica, o *stocastiche* se i tempi sono definiti in modo probabilistico [2].

Le reti di Petri colorate includono un insieme finito di *colori* i quali rappresentano un insieme di tipi di dato, di operazioni e di funzioni che caratterizzano un elemento della rete di Petri¹.

Nonostante la semplicità di rappresentazione ed utilizzo, l'analisi del comportamento di una rete di Petri può risultare particolarmente complessa a causa del problema della *state explosion*. Rappresentare esplicitamente tutte le possibili sequenze di esecuzione di una rete può diventare un compito impossibile: il numero di stati raggiungibili cresce almeno in maniera **esponenziale** rispetto al numero di componenti del sistema.

Per affrontare questa problematica è stato introdotto il concetto di "prefisso dell'unfolding": un grafo aciclico e orientato, detto *occurrence net*, che rappresenta tutti i possibili comportamenti della rete originale in termini di relazioni causali, di concorrenza e di conflitto. L'unfolding espande la rete di

¹<https://www2.informatik.uni-hamburg.de/TGI/PetriNets/classification/level3/CPN.html>

Petri, rendendo esplicite tutte le possibili esecuzioni della stessa, ed il prefisso di un unfolding ne semplifica l'analisi e lo studio.

Il primo capitolo di questo elaborato include una spiegazione generale del concetto di rete di Petri e *branching process*, utili e necessari per introdurre il concetto di unfolding. Nel secondo capitolo viene analizzato più a fondo il concetto di unfolding e di prefisso finito e completo dell'unfolding, procedendo con una descrizione dell'algoritmo *ERV*, da Esparza-Römer-Vogler, i cognomi degli autori dell'articolo [3], un algoritmo in grado di generare un prefisso finito e completo dell'unfolding di una rete di Petri. Inoltre, in tale capitolo è presente l'analisi di miglioramenti di tipo euristico su questo algoritmo. Nel terzo capitolo viene analizzata la tecnica del Directed Unfolding, una tecnica per dirigere la costruzione del prefisso di un unfolding tramite la ricerca di un obiettivo [4]. Nel quarto, ed ultimo, capitolo viene esposta una personale implementazione software di unfolding sviluppata durante lo stage svolto in università.

Capitolo 1

Definizioni preliminari

In questo capitolo verranno definite le nozioni base di rete di Petri, e successivamente i concetti di branching process, di configurazione e di taglio.

1.1 Rete di Petri

Le definizioni formali sulle reti di Petri sono riprese da [3].

Una rete è una tripla (P, T, W) dove:

- $P = \{p_1, p_2, \dots, p_m\}$ è un insieme finito di posti
- $T = \{t_1, t_2, \dots, t_n\}$ è un insieme finito di transizioni
- $W : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$ è una funzione peso che indica la presenza, o meno, di un arco da un posto ad una transizione o da una transizione ad un posto ed il numero di token che tale arco consuma durante l'occorrenza di una transizione.

Posti e transizioni sono chiamati *nodi*. Se $W(x, y) > 0$ allora si dice che è presente un arco da x a y . Una rete si può considerare come un grafo orientato. Un *path* è una sequenza non vuota di nodi, tale che un arco è presente da qualunque nodo al seguente.

I posti sono disegnati tramite dei cerchi, rappresentano delle *condizioni* o degli *stati* del modello che si vuole rappresentare. Possono contenere dei *token*, o marche, ovvero dei simboli che indicano se tale condizione sia *vera* o se una risorsa è disponibile. Le transizioni sono invece disegnate tramite rettangoli e rappresentano *eventi* o *azioni* che modificano lo stato del sistema, *consumando* uno, o più, token dai posti precedenti e producendo token per i posti successivi. Posti e transizioni sono infine collegati da archi orientati, che rappresentano la relazione tra di loro. Un posto non può essere in relazione con un altro posto, così come una transizione non può essere in relazione con un'altra transizione [3].

La precedente è una definizione generale di rete di Petri, corrispondente alle reti dette *ordinarie*. Nel seguito di questo elaborato, viene assunto che il peso degli archi della rete non possa eccedere il valore di 1. La funzione peso considerata sarà quindi $W : (P \times T) \cup (T \times P) \rightarrow \{0, 1\}$.

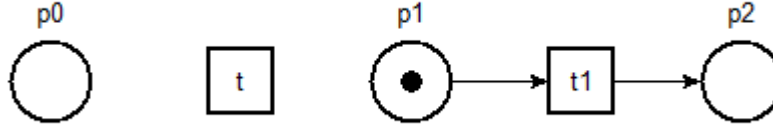


Figura 1.1: Un posto, una transizione e un *path*

Il *preset* di un nodo x , indicato con $\bullet x$, è l'insieme $\{y \in P \cup T \mid W(y, x) = 1\}$.

Il *postset* di x , indicato con $x^\bullet$, è l'insieme $\{y \in P \cup T \mid W(x, y) = 1\}$.

Preset e *postset* rappresentano gli elementi precedenti, e successivi, di un posto o di una transizione.

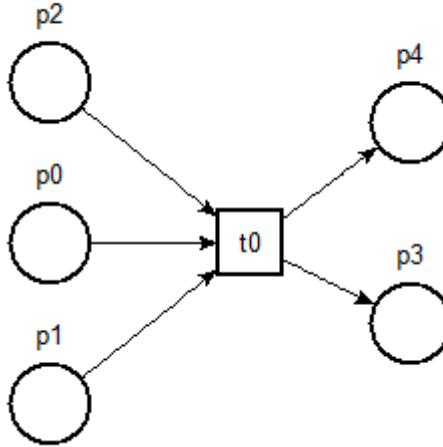


Figura 1.2: Una transizione t_0 con il suo *preset* $\{p_0, p_1, p_2\}$ e *postset* $\{p_3, p_4\}$

Una *marcatore* di una rete (P, T, W) è una funzione $M : P \rightarrow N$, dove N indica l'insieme dei numeri naturali incluso lo 0. M è il multiinsieme contenente $M(p)$ copie di p , per ogni $p \in P$. Se $P = \{p_1, p_2\}$ e $M(p_1) = 1, M(p_2) = 2$, si scrive $M = \{p_1, p_2, p_2\}$. La marcatura è una funzione che indica il numero di *token* presenti in un posto [3].

Una quadrupla $\Sigma = (P, T, W, M_0)$ è una *rete di Petri*, o un *sistema di rete* (*net system* in numerose pubblicazioni in lingua inglese come [3] o [5]), se:

- (P, T, W) è una rete
- M_0 è una marcatura di (P, T, W) , chiamata *marcatura iniziale* di Σ

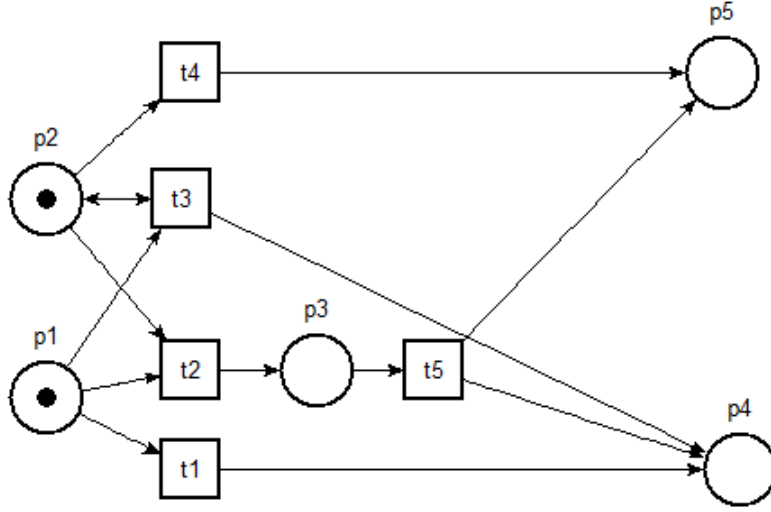


Figura 1.3: Una rete di Petri, i posti $p1$ e $p2$ hanno un token [6]

Una marcatura M *abilita* una transizione t , se ogni posto $p \in \bullet t$ ha un *token*, ovvero se $M(p) > 0$ per ogni $p \in \bullet t$. In altre parole, ogni posto precedente alla transizione t , deve avere un token per poterla abilitare. Se t è abilitata su M , allora può *occorrere* o *scattare*, e questa occorrenza porterà ad una nuova marcatura $M'(p) = M(p) - W(p, t) + W(t, p)$, ottenuta rimuovendo un token da ogni posto nel preset di t , e aggiungendo un token in ogni posto del suo postset. Si definisce questa relazione come $M \xrightarrow{t} M'$. Nella figura 1.3 le transizioni abilitate sono $\{t_1, t_2, t_3, t_4\}$, in quanto tutti gli elementi dei loro *preset* hanno un *token*. Una sequenza di transizioni $\sigma = t_1 t_2 \dots t_n$ è una *sequenza di occorrenze* se esistono marcature $M_1, M_2, \dots, M_n$ tali che M_n è la marcatura raggiunta dall'occorrenza di σ , e viene indicata con $M_0 \xrightarrow{\sigma} M_n$. M è una *marcatura raggiungibile* se esiste una sequenza di occorrenze tali che $M_0 \xrightarrow{\sigma} M_n$, ovvero se $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} M_n$ [3].

Una marcatura M di una rete si dice *n-safe* se $M(p) \leq n$ per ogni posto p , ovvero, se il numero di token presenti in ogni posto di una marcatura non eccede un valore n , allora tale M è *n-safe*. Una rete Σ si dice *n-safe* se tutte le sue marcature raggiungibili sono *n-safe* [3].

L'occorrenza della transizione t_2 porterebbe ad una nuova marcatura, dove p_1 e p_2 vedrebbero i loro token "consumati", abilitando di conseguenza la transizione t_5 in quanto p_3 avrebbe un token, ciò si può vedere nella figura 1.4.

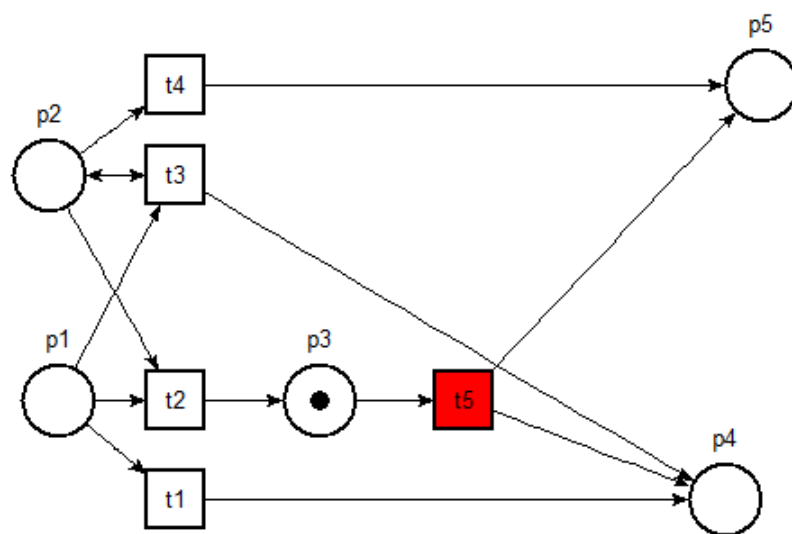


Figura 1.4: Occorrenza della transizione t_2 . La transizione t_5 è ora abilitata

1.2 Branching Processes

Verranno ora definiti i *branching processes*. Per poterlo fare è necessario definire prima il concetto di *reti di occorrenze* (*occurrence net*).

Occurrence Net [3] Per definire le reti di occorrenze, bisogna prima definire le relazioni di *causalità*, di *conflitto* e di *concorrenza* tra i nodi di una rete.

- Due nodi x e y , sono in *relazione causale*, indicato con $x < y$, se la rete contiene un path, con almeno un arco, che va da x a y .

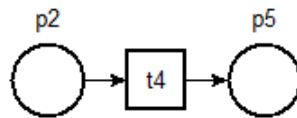


Figura 1.5: Relazione di causalità $p_2 < p_5$

- x e y sono in *conflitto*, indicato con $x \# y$, se nella rete sono presenti due path $pt_1 \dots x_1$ e $pt_2 \dots x_2$ con lo stesso posto iniziale p e tale che $t_1 \neq t_2$.

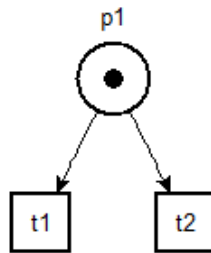


Figura 1.6: Due transizioni in conflitto $t_1 \# t_2$

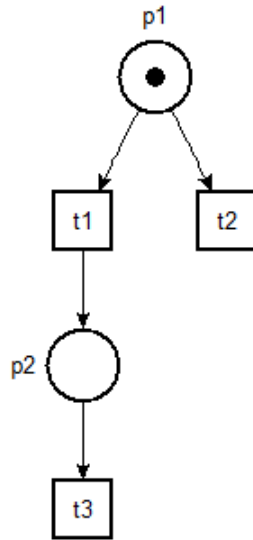


Figura 1.7: Il conflitto è ereditato secondo le relazioni causali. Risulta quindi che $t_3 \# t_2$ in quanto $t_1 < t_3$

- x e y sono *concorrenti* se non sono né in conflitto né in relazione causale, si indica con $x || y$.

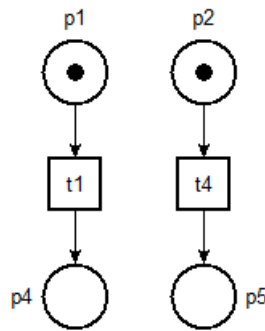


Figura 1.8: Due transizioni concorrenti $t_1 || t_4$

Una *rete di occorrenze* è una rete $O = (B, E, F)$ tale che [3]:

1. $|\bullet b| \leq 1$ per ogni $b \in B$;
2. O è una rete aciclica;
3. per ogni $x \in B \cup E$, si ha l'insieme di elementi $y \in B \cup E$ tale che $y < x$ è finito;
4. nessun elemento è in conflitto con sé stesso.

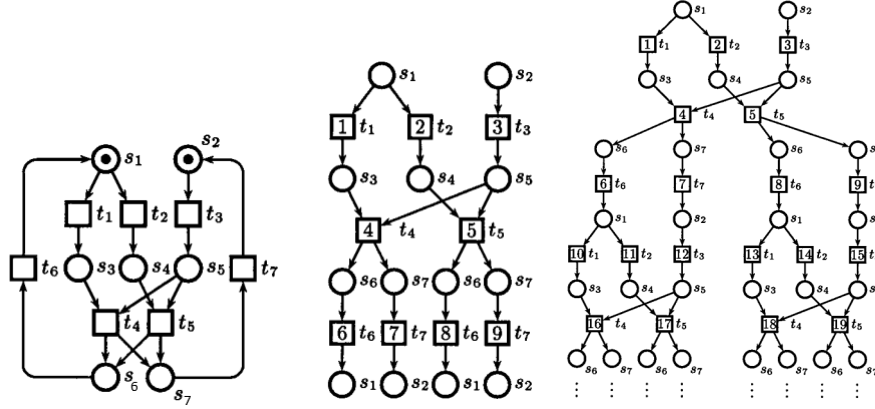


Figura 1.9: Una rete di Petri e due dei suoi branching processes [3]

Gli elementi B ed E sono chiamati rispettivamente *condizioni* ed *eventi*.

$Min(O)$ indica l'insieme di elementi minimi di $B \cup E$, ovvero gli elementi iniziali, con un preset vuoto.

Branching processes Le reti di occorrenze ottenute dagli unfolding di una rete, sono chiamate branching processes, così definite [3][7]:

Un *branching process* di una rete $N = (P, T, W, M_0)$, è una rete di occorrenze $\beta = (B, E, F, h)$, dove h è una funzione di etichettatura $h : B \cup E \rightarrow P \cup T$ che soddisfa le seguenti proprietà [3]:

1. $h(B) \subseteq P$ e $h(E) \subseteq T$, ovvero la natura dei nodi è preservata;
2. per ogni $e \in E$, $h\{\bullet e\} = \bullet h(e)$ e $h\{e\bullet\} = h(e)\bullet$, ovvero h mantiene invariata la struttura di input e output dei nodi
3. $h\{Min(\beta)\} = M_0$, ovvero β ha come radice la marcatura iniziale M_0 ;
4. per ogni $e_1, e_2 \in E$, se $\bullet e_1 = \bullet e_2$ e $h(e_1) = h(e_2)$, allora $e_1 = e_2$, ovvero β non duplica le transizioni di N .

Per ogni rete N , esiste un unico branching process massimale, chiamato appunto *unfolding* di N [5].

In [3], viene affermato che i *branching processes* differiscono tra di loro in base a quanto unfolding della rete riescono a produrre, è quindi utile definire una *relazione di prefisso*, per formalizzare il concetto che "un branching process può fare meno *unfolding* di un altro".

Siano $\beta' = (O', h')$ e $\beta = (O, h)$ due branching processes di una rete di Petri. Allora β' è un *prefisso* di β se O' è una sotto rete di O che soddisfa le seguenti proprietà: [3]

- $Min(O)$ appartiene ad O'
- Se la condizione b appartiene ad O' , allora gli eventi di input di $e \in \bullet b$ in O appartengono anche a O'

- Se un evento e appartiene ad O' , allora le sue condizioni di input e output $\bullet e \cup e \bullet$ in O appartengono anche ad O'

1.3 Configurazioni e tagli

Configurazione Una configurazione C di un branching process è un insieme di eventi che soddisfa due condizioni:

- $\forall e, e' \in C : \neg(e \# e')$
- $e \in C \Rightarrow \forall e' \leq e : e' \in C$

Ovvero: tutti gli eventi nella configurazione C non sono in conflitto, si dice che C è *conflict-free*, e, dato un evento e nella configurazione C , ogni suo evento causalmente precedente e' fa parte della configurazione C , si dice che C è *causally closed* [3].

Nella figura 1.9, l'insieme di eventi $\{1, 3, 4, 6\}$ è una configurazione, mentre gli insiemi $\{3, 4\}$ e $\{1, 2\}$ non lo sono: il primo non è causally closed e il secondo non è conflict-free. Si può facilmente notare come una configurazione sia l'insieme di eventi che possono occorrere a partire dalla marcatura iniziale: esiste una sequenza di transizioni, a partire da M_0 , dove ogni evento nell'insieme della configurazione accade una volta sola: per $\{1, 3, 4, 6\}$ accadono prima 1 e 3, successivamente 4 e 6, mentre né $\{3, 4\}$ né $\{1, 2\}$ potrebbero mai accadere in sequenza a partire dalla marcatura iniziale [7].

Viene di seguito introdotto il concetto di *configurazione locale* di un evento e , in modo da potervi associare la sua marcatura raggiungibile $Mark([e])$:

La configurazione locale di un evento e di un branching process, indicata con $[e]$, è l'insieme di eventi e' tali che $e' \leq e$ [3].

Ovvero, una configurazione locale di un evento dell'unfolding, rappresenta la "catena" di eventi che portano a quel determinato evento [3].

Taglio Un insieme di condizioni di un branching process è un *co-set* se i suoi elementi sono in relazione di concorrenza (indicata con *co*) a coppie, ovvero, come affermato in [6]: un insieme di condizioni B' , tale che per tutte le condizioni distinte $b, b' \in B'$ si ha che $b \text{ co } b'$, allora si chiama *co-set*. Un *co-set* massimale rispetto all'inclusione degli insiemi è detto *taglio* [3]. Nel branching process più piccolo della figura 1.9, l'insieme contenente le condizioni di output degli eventi 6 e 4 è un *taglio*.

Una marcatura M di un sistema Σ è rappresentata in un branching process $\beta = (O, h)$ di Σ , se β contiene un taglio c tale che, per ogni posto $h \in \Sigma$, c , contiene esattamente $M(p)$ condizioni b con $h(b) = p$ [3]. Ad esempio, nella figura 1.9, la marcatura $\{s_1, s_7\}$ è rappresentata grazie al taglio prima menzionato.

Le configurazioni finite e i tagli, sono in stretta relazione. Sia C una configurazione finita del branching process $\beta = (O, h)$. Allora il *co-set* chiamato $Cut(C)$ è un taglio, definito come:

$$Cut(C) = (Min(O) \cup C \bullet) \setminus \bullet C$$

Ovvero, data una configurazione C , l'insieme di posti $Cut(C)$ rappresenta una marcatura raggiungibile, indicata con $Mark(C)$. Si può dire che $Mark(C)$ è la marcatura raggiungibile facendo scattare una configurazione C . Nel branching process in figura 1.9, si ha che $Mark([1, 3, 4, 6]) = \{s_1, s_7\}$ [3].

Capitolo 2

Algoritmi per la costruzione di prefissi finiti

In questo capitolo verrà presentato un algoritmo per la costruzione di prefissi finiti dell'unfolding di una rete di Petri. In particolare verrà trattato l'algoritmo *ERV* (Esparza-Römer-Vogler) formulato in [3], verranno esposti i concetti di *ordine adeguato* per le configurazioni dell'unfolding e di *eventi cut-off*. Infine verrà fornito lo pseudocodice di tale algoritmo, sempre forniti in [3].

2.1 Costruzione dell'unfolding

Come già anticipato nel capitolo precedente, un branching process di una rete di Petri è composto da due elementi principali: un insieme di *condizioni* ed un insieme di *eventi*. Entrambi questi insiemi servono per comporre l'unfolding di una rete di Petri Σ , sotto forma di un insieme $\{n_1, n_2, \dots, n_k\}$ di *nodi*. Ed è quindi necessario definire delle strutture dati adeguate per Condizioni ed Eventi [3]:

- Una *Condizione* presenta due campi: il posto di Σ rappresentato dalla condizione ed un puntatore all'evento che ha "generato" tale condizione. Nel caso si tratti di una condizione *iniziale*, allora il puntatore all'evento sarà *NIL*.
- Anche un *Evento* presenta due campi: una transizione di Σ , rappresentata dall'evento, e un puntatore ad un insieme di condizioni, ovvero quelle che hanno abilitato tale evento.

Una condizione è rappresentata dalla coppia (p, e) , o (p, NIL) nel caso di condizione iniziale, dove p è un posto ed e è l'evento che la genera, mentre un evento è rappresentato dalla coppia (t, X) , dove t è la transizione e X è l'insieme di Condizioni che hanno generato l'evento in considerazione.

Considerato che la costruzione dell'unfolding è un processo iterativo, che genera un grafo aggiungendo ad esso, in ogni iterazione, un evento con le sue condizioni di output, è necessario definire il concetto di "evento che si può aggiungere ad un dato branching process" [3]. In altre parole, un evento "successivo" a quelli già inseriti nell'unfolding. Data una transizione t di una rete di Petri Σ ,

con un insieme di posti di output $\{p_1, \dots, p_n\}$, allora la coppia $e = (t, X)$, dove X è un *co-set* di condizioni, ovvero un insieme di condizioni concorrenti, è una *possibile estensione* (possible extension) di un branching process se, aggiunto questo evento ed i suoi output all'insieme di nodi $\{n_1, \dots, n_k\}$, si ottiene sempre un branching process [3]. L'insieme di possibili estensioni di un branching process β si indica con $PE(\beta)$. In particolare, $PE(\beta)$ indica anche la funzione di ricerca delle possibili estensioni nell'algoritmo di unfolding, funzione non fornita esplicitamente in [3].

In [3] viene quindi data una definizione più formale di una possibile estensione:

Possibile estensione Sia β un branching process di una rete di Petri Σ . Le possibili estensioni di β sono le coppie (t, X) , dove X è un *co-set* di condizioni, ovvero un insieme di condizioni concorrenti, di β e t è una transizione di Σ , tale che:

- $p(X) = \bullet t$, ovvero le etichette a cui si riferiscono le condizioni di X rappresentano i posti precedenti alla transizione t
- (t, X) non è già parte di β

L'algoritmo inizia con l'unfolding contenente solo le condizioni della marcatura iniziale di Σ . Gli eventi e le loro condizioni di output vengono aggiunti ad ogni iterazione, ed un posto può comparire più volte.

Viene di seguito proposto l'algoritmo di unfolding base: in input riceve una rete di Petri $\Sigma = (N, M_0)$, dove $M_0 = \{p_1, \dots, p_n\}$ sono i posti iniziali, e fornisce in output l'unfolding di Σ :

Algoritmo 1 Algoritmo di unfolding [3]

```

1:  $Unf := \{(p_1, \emptyset), \dots, (p_n, \emptyset)\};$ 
2:  $pe := PE(Unf);$ 
3: while  $pe \neq \emptyset$  do
4:   aggiunge ad  $Unf$  un evento  $e = (t, X)$  ed una condizione  $(p, e)$ 
5:   per ogni posto di output  $p$  di  $t$ ;
6:    $pe := PE(Unf)$ 
7: end while
```

L'algoritmo appena descritto genera un unfolding a partire da una rete di Petri. Nel caso generale, corrispondente al precedente pseudocodice, è un algoritmo infinito, termina solamente se la rete Σ , di cui si vuole produrre un unfolding, non ha sequenze di occorrenze infinite, ovvero non presenta cicli al suo interno. Ad ogni iterazione, Unf viene espanso e si cercano possibili estensioni a partire da Unf stesso [3].

2.2 Costruzione di prefissi finiti completi

Dal momento che l'unfolding di una rete di Petri potrebbe essere infinito, è opportuno considerare la costruzione di solamente una sua parte iniziale: un *prefisso finito completo* dell'unfolding, che contiene tutte le informazioni disponibili in un unfolding "intero". Un prefisso si può ottenere perché una rete

presenta una quantità finita di marcature raggiungibili, che, ad un certo punto durante la costruzione, iniziano a ripresentarsi a causa della natura ciclica di determinate reti di Petri di cui si vuole costruire l'unfolding, rendendo ridondante l'aggiunta di altri elementi già inseriti in precedenza [3].

Un prefisso β' di un unfolding β è detto *completo* se, per ogni marcatura raggiungibile M [4]:

- esiste una configurazione $C \in \beta'$ la cui marcatura $Mark[C]$ è uguale ad M , ovvero la marcatura M è già presente all'interno del prefisso β' ;
- per ogni transizione t abilitata dalla marcatura M , esiste una configurazione $C \cup \{e\}$ tale che: l'evento e non fa parte della configurazione C ed e viene etichettato da t , ovvero l'evento e viene abilitato in $Cut(C)$ [8]

Un prefisso è ottenuto troncando la costruzione dell'unfolding, trovando degli eventi in cui è possibile fermare il processo di costruzione senza perdere informazione. Questi eventi sono chiamati *eventi cut-off* (cut-off events) e non contribuiscono all'ottenimento di nessuna nuova marcatura raggiungibile [8]. Gli eventi cut-off sono definiti in termini di *ordini adeguati* sulle configurazioni.

Data una configurazione C , si considera $C \oplus E$ una configurazione che estende C con un insieme finito di eventi E che non ha elementi in comune con la configurazione C , allora:

Ordine Adeguato [8] Un ordine parziale $\prec$, su configurazioni finite dell'unfolding di una rete di Petri, è detto adeguato se:

- $\prec$ è un ordine ben fondato, ovvero non esistono catene discendenti infinite di elementi,
- $C_1 \subset C_2$ implica che $C_1 \prec C_2$,
- $\prec$ è mantenuto da estensioni finite: per ogni coppia di configurazioni C_1 e C_2 tali che $C_1 \prec C_2$ e $Mark(C_1) = Mark(C_2)$, allora, per tutte le estensioni finite $C_1 \oplus E_1$ e $C_2 \oplus E_2$, tali che E_1 ed E_2 sono strutturalmente simili, si ha che $C_1 \oplus E_1 \prec C_2 \oplus E_2$, ovvero, se una configurazione è "più piccola" di un'altra, secondo l'ordine $\prec$, l'ordine non viene alterato da estensioni finite con la stessa struttura. In altre parole, se due configurazioni C_1 e C_2 descrivono lo stesso stato, aggiungere a queste configurazioni delle estensioni strutturalmente uguali non cambia l'ordine $\prec$.

In [9] la dimensione delle configurazioni locali si basa sul numero di eventi contenuti in esse: $[e']$ è più piccola di $[e]$ se contiene meno eventi, ovvero $|[e']| < |[e]|$. Questo è un ordine adeguato. Diventa quindi possibile terminare l'unfolding di una rete a partire da un evento e se questo evento porta ad una marcatura già ottenuta da un altro evento e' , tale che $[e'] \prec [e]$, dato che gli elementi ottenuti a partire da e saranno gli stessi ottenuti da e' [8]. Si può definire formalmente il concetto di eventi cut-off:

Evento cut-off [3] Sia $\prec$ un ordine adeguato sulle configurazioni dell'unfolding di una rete di Petri. Sia β un prefisso dell'unfolding contenente un evento e . L'evento e si dice *evento cut-off* di β , relativamente all'ordine $\prec$, se β contiene una configurazione locale $[e']$ tale che:

- $Mark[e] = Mark[e']$,
- $[e'] \prec [e]$

Come precedentemente affermato, se un evento è strutturalmente simile ad un evento già presente nell'unfolding, la sua aggiunta alla rete di occorrenze risulta essere ridondante ed è possibile terminare il processo di costruzione.

Viene di seguito proposto l'algoritmo **ERV** (Esparza-Römer-Vogler), fornito in [3], per la costruzione di un prefisso finito completo dell'unfolding di una rete di Petri. In input riceve una rete di Petri $\Sigma = (N, M_0)$ e fornisce in output un prefisso finito completo Fin dell'unfolding.

Algoritmo 2 Algoritmo di costruzione del prefisso finito completo [3]

```

1:  $Fin := (p_1, \emptyset), \dots, (p_k, \emptyset)$ ;
2:  $pe := PE(Fin)$ ;
3:  $cut-off := \emptyset$ ;
4: while  $pe \neq \emptyset$  do
5:   prende un evento  $e = (t, X)$  in  $pe$  tale che  $[e]$  è
6:   minimo rispetto all'ordine  $\prec$ ;
7:   if  $[e] \cap cut-off = \emptyset$  then
8:     accoda a  $Fin$  l'evento  $e$  ed una condizione  $(p, e)$ 
9:     per ogni posto di output  $p$  di  $t$ ;
10:     $pe := PE(Fin)$ ;
11:    if  $e$  è un evento cut-off di  $Fin$  then
12:       $cut-off := cut-off \cup \{e\}$ ;
13:    end if
14:  else
15:     $pe := pe \setminus \{e\}$ 
16:  end if
17: end while

```

Si può notare come l'evento scelto dalle possibili estensioni venga effettivamente inserito nel prefisso, insieme alle sue condizioni di output, solo se tale evento non è di tipo cut-off. Sempre e solo in questo caso vengono trovate le nuove possibili estensioni, e viene controllato se l'evento appena inserito sia esso stesso un cut-off del prefisso: in caso affermativo viene inserito in un insieme apposito di eventi cut-off. In caso contrario l'evento scelto viene tolto dall'insieme di possibili estensioni, questa operazione permette alla coda di estensioni di svuotarsi, portando a termine la costruzione del prefisso.

Il prefisso Fin è finito proprio grazie al concetto di taglio e di evento cut-off: l'algoritmo riconosce quando un taglio, ovvero una marcatura raggiungibile da una sequenza di eventi, è stato già inserito all'interno di Fin , segnando gli eventi che portano a marcature già raggiunte in precedenza come eventi cut-off. Dato che il numero di posti e transizioni di una rete Σ è finito, ne consegue che il numero di eventi e condizioni inseribili nel prefisso è finito, ne risulta che quest'ultimo sia finito [3].

Fin risulta essere completo per due motivi: ogni marcatura raggiungibile di Σ è rappresentata in Fin , grazie al concetto di evento cut-off introdotto precedentemente. Inoltre ogni transizione t , che può accadere nella rete Σ è rappresentata in Fin da un evento etichettato da t [3].

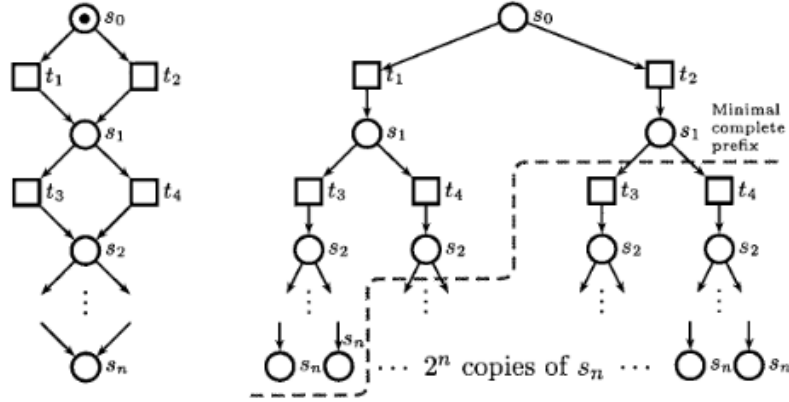


Figura 2.1: Un particolare tipo di rete di Petri con il suo unfolding [3]

2.3 Un ordine adeguato per reti di Petri

L'ordine adeguato utilizzato in [9], dove le configurazioni si differenziano e confrontano in base al numero di eventi presenti al loro interno, può essere molto inefficiente [3]. Un caso estremo risulta quello presente nella figura 2.1: la dimensione del prefisso completo risulta essere $O(n)$, ma il branching process, generato dall'algoritmo con questo tipo di ordine adeguato, risulta essere di dimensione $O(2^n)$ [3]. Questo accade perché, per ogni marcatura M , tutte le configurazioni locali $[e]$ che soddisfano $Mark([e]) = M$, ovvero che raggiungono una stessa marcatura, hanno la stessa dimensione, quindi l'ordine definito in [9] non è in grado di far identificare all'algoritmo delle configurazioni "più piccole" di altre, di conseguenza non viene trovato nessun evento cut-off, continuando a generare un prefisso che cresce esponenzialmente [3].

Siccome ordini adeguati differenti portano alla generazione di prefissi finiti differenti, non è necessario cambiare l'algoritmo di costruzione di un prefisso finito ma solamente l'ordine adeguato utilizzato. In particolare, il nuovo ordine adeguato presentato in [3] introduce una nozione di evento cut-off *più debole* rispetto a quella descritta precedentemente. Con questa nuova definizione, un evento cut-off rispetto l'ordine definito da McMillan in [9] è un evento cut-off rispetto al nuovo ordine adeguato, ma non è sempre vero il contrario [3]. L'algoritmo 2, che utilizza il nuovo ordine, genera *almeno* tanti eventi cut-off quanti quelli generati seguendo l'ordinamento definito da McMillan, in [9], ed in alcuni casi anche di più ottenendo un prefisso completo più piccolo [3].

Sia quindi $\Sigma = (N, M_0)$ una rete di Petri, e sia $\ll$ un ordine totale sulle transizioni di Σ . Dato un insieme E di eventi, sia $\varphi(E)$ una sequenza di transizioni ordinata secondo $\ll$, e che contenga una transizione t della rete, per ogni evento in E che etichetti tale transizione t . Se la sequenza è $t_1 \ll t_2 \ll t_3 \ll t_4$, allora l'insieme E contiene quattro eventi etichettati da t_1, t_2, t_3, t_4 , allora $\varphi(E) = t_1 t_2 t_3 t_4$. Risulta che $\varphi(E_1) \ll \varphi(E_2)$ se $\varphi(E_1)$ è *lessicograficamente* più piccolo di $\varphi(E_2)$ rispetto l'ordine $\ll$ [3].

Per esempio, considerata una sequenza ordinata di transizioni $t_1 < t_2 < t_3 < t_4$ e prendendo due configurazioni $C_1 = \{e_1, e_2, e_3\}$ e $C_2 = \{e_4, e_5, e_6\}$ di

dimensione uguale e con la stessa marcatura raggiunta, si ha che $C_1 = \{p(e_1) = t_1, p(e_2) = t_2, p(e_3) = t_4\}$ e $C_2 = \{p(e_4) = t_1, p(e_5) = t_3, p(e_6) = t_3\}$. Risulta quindi che $\varphi(C_1) = t_1 t_2 t_4$ e $\varphi(C_2) = t_1 t_3 t_3$. Viene quindi svolto il confronto lessicografico seguendo l'ordine delle posizioni:

1. Prima posizione: $t_1 = t_1$, risultano uguali, quindi si può passare alla posizione successiva;
2. Seconda posizione: $t_2 < t_3$, il confronto si ferma.

Si conclude che $\varphi(C_1) < \varphi(C_2)$.

Si può definire l'ordine parziale $\prec_E$

Ordine parziale $\prec_E$ [3] Siano C_1 e C_2 due configurazioni dell'unfolding di una rete di Petri. $C_1 \prec_E C_2$ vale se:

- $|C_1| < |C_2|$, oppure
- $|C_1| = |C_2|$, e $\varphi(C_1) \ll \varphi(C_2)$.

Se $\prec_E$ viene utilizzato come ordine adeguato, il prefisso completo generato dall'algoritmo 2 corrisponde alla rete prima della linea tratteggiata nella figura 2.1.

Per ottenere un ordine adeguato che garantisca che la grandezza di un prefisso completo sia almeno quanto quella del grafo di marcature raggiungibili di una rete di Petri, viene introdotto in [3] un ordine adeguato *totale*: ogni volta che un evento e viene generato successivamente ad un altro evento e' , tale che $Mark([e]) = Mark([e'])$, si ha che $[e'] \prec [e]$, dato che gli eventi sono generati seguendo l'ordine totale $\prec$, e quindi e viene segnato come evento cut-off. Gli ordini adeguati totali presentano la seguente proprietà:

Una proprietà degli ordini adeguati totali [3] Sia $\prec$ un ordine adeguato totale, sia Fin l'output dell'algoritmo 2 con la rete di Petri Σ in input. Il numero di eventi non cut-off di Fin non supera il numero di marcature raggiungibili di Σ [3].

L'ordine $\prec_E$ non è totale. Prendiamo in considerazione la seguente rete con il suo unfolding:

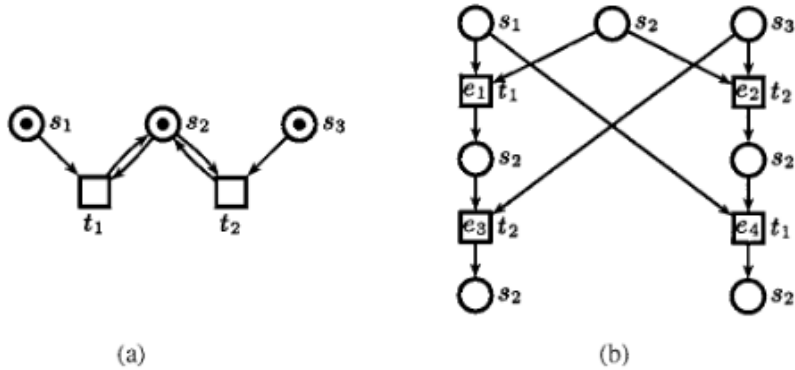


Figura 2.2: Una rete 1-safe (a) ed il suo unfolding (b) [3]

Le configurazioni $C_1 = \{e_1, e_3\}$ e $C_2 = \{e_2, e_4\}$ hanno una dimensione di 2, e, assumendo che $t_1 \ll t_2$, si ha che $\varphi(C_1) = t_1 t_2 = \varphi(C_2)$. Quindi non si ha né $C_1 \prec_E C_2$ né $C_2 \prec_E C_1$.

2.4 Un ordine totale per reti 1-safe

Una rete 1-safe è un particolare tipo di rete: ogni posto può contenere al massimo un solo token per ogni marcatura raggiungibile. Sia $\Sigma = (N, M_0)$ una rete, e $\ll$ un ordine totale sulle transizioni di Σ . In [3] viene introdotta la *forma normale Foata* per una configurazione. Data una configurazione finita C , la sua forma normale Foata FC è la lista di insiemi di eventi costruiti tramite il seguente algoritmo [3]:

Algoritmo 3 Algoritmo per la costruzione della forma normale Foata di una configurazione [3]

```

1:  $FC = \emptyset$ ;
2: while  $C \neq \emptyset$  do
3:   aggiunge  $Min(C)$  a  $FC$ ;
4:    $C = C \setminus Min(C)$ ;
5: end while
```

La forma normale di Foata si ottiene suddividendo ripetutamente l'insieme degli eventi minimi.

Date due configurazioni C_1 e C_2 , le loro forme normali di Foata $FC_1 = C_{1l} \dots C_{1n_1}$ e $FC_2 = C_{2l} \dots C_{2n_2}$ possono essere confrontate, con rispetto all'ordine $\ll$: si dice che $FC_1 \ll FC_2$ se esistono $1 \leq i \leq n_1$ tali che:

- $\varphi(C_{1j}) = \varphi(C_{2j})$ per ogni $1 \leq j < i$, e
- $\varphi(C_{1i}) \ll \varphi(C_{2i})$.

Si può procedere con il definire l'ordine $\prec_F$:

Ordine $\prec_F$ [3] Siano C_1 e C_2 due configurazioni dell'unfolding di una rete di Petri. $C_1 \prec_F C_2$ vale se:

- $|C_1| < |C_2|$, oppure
- $|C_1| = |C_2|$ e $\varphi(C_1) \ll \varphi(C_2)$, oppure
- $\varphi(C_1) = \varphi(C_2)$ e $FC_1 \ll FC_2$.

In altre parole, per decidere se $C_1 \prec_F C_2$, si svolgono 3 "controlli": prima si confronta la dimensione, tramite il numero di eventi nelle due configurazioni; se sono di dimensione uguale, si svolge il confronto lessicografico tra $\varphi(C_1)$ e $\varphi(C_2)$; se anche a seguito di questo confronto si ottiene lo stesso risultato, si confrontano FC_1 e FC_2 .

$\prec_F$ è un ulteriore raffinamento dell'ordine $\prec_E$. L'ordine $\prec_F$ è sia adeguato che totale, questo grazie ad una fondamentale proprietà delle reti 1-safe:

Proprietà delle reti 1-safe [3] Qualsiasi coppia di condizioni concorrenti del branching process di una rete 1-safe ha etichette differenti.

2.5 Trovare possibili estensioni

Trovare le possibili estensioni da aggiungere durante la costruzione del prefisso dell'unfolding è un processo fondamentale e computazionalmente oneroso, risultando in molti casi il processo più lento della costruzione del prefisso [6]. Numerosi studi si sono concentrati su come rendere più efficiente questa operazione come in [6] e [10], fornendo dei possibili passi ed euristiche per trovare eventi successivi inseribili nel prefisso in maniera efficiente.

In particolare, in [6] si dimostra come si possono trovare delle possibili estensioni, non controllando una per una ogni transizione della rete di Petri originale, ma controllandole tutte in una volta, unendo parti comuni del processo di ricerca delle possibili estensioni.

Algoritmo semplificato per la generazione di prefissi

Nell'algoritmo 2, viene impiegata, ma non esplicitamente descritta, una generica funzione chiamata $PE(Fin)$ per trovare le possibili estensioni del prefisso in costruzione, che, come accennato in precedenza, risulta essere la componente più "complicata" dell'algoritmo. Uno dei metodi utilizzati per ridurre la complessità di questa funzione è quello di unire parti comuni del processo di individuazione e inserimento delle istanze di transizioni nel prefisso dell'unfolding.

[6] propone questa metodologia dove, invece di aggiornare l'intero insieme di possibili estensioni ad ogni passo del ciclo principale, aggiorna l'insieme pe , già ottenuto in precedenza, aggiungendo solo gli eventi successivi ad un evento e , ovvero gli eventi che si ottengono "consumando" le condizioni presenti in $e^\bullet$, dove e risulta essere l'ultimo evento inserito nel prefisso.

Seguendo questa metodologia, l'insieme pe viene visto come una *coda a priorità*, con gli eventi ordinati, sulle loro configurazioni locali, secondo l'ordine adeguato $\prec$. Nel corpo del *while* dell'algoritmo 2, la funzione $UpdatePotExt(Fin, e)$ prende il posto della chiamata a funzione $PE(Fin)$: in questo modo, invece di trovare tutte le possibili estensioni del prefisso, vengono ricercate le (Fin, e) – *estensioni* [6], ovvero le possibili estensioni del solo evento e preso in considerazione. Grazie a questa semplificazione, c'è la sicurezza di non dover controllare le possibili estensioni di un particolare evento più di una volta, di conseguenza è possibile inserire gli eventi del tipo cut-off alla fine dell'algoritmo, evitando di dover controllare se una configurazione contiene eventi cut-off [6].

Di seguito l'algoritmo per la generazione di un prefisso con la nuova funzione $UpdatePotExt(Fin, e)$:

Algoritmo 4 Algoritmo di costruzione del prefisso finito completo aggiornato [6]

```

1:  $Fin := (p_1, \emptyset), \dots, (p_k, \emptyset);$ 
2:  $pe := PE(Fin);$ 
3:  $cut-off := \emptyset;$ 
4: while  $pe \neq \emptyset$  do
5:   prende un evento  $e = (t, X)$  in  $pe$  tale che  $[e]$  è
6:   minima rispetto all'ordine  $\prec$ ;
7:    $pe := pe \setminus \{e\};$ 
8:   if  $e$  è un evento cut-off di  $Fin$  then
9:      $cut-off := cut-off \cup \{e\};$ 
10:  else
11:    accoda a  $Fin$  l'evento  $e$  ed una condizione  $(p, e)$ 
12:    per ogni posto di output  $p$  di  $t$ ;
13:     $pe := pe \cup UpdatePotExt(Fin, e);$ 
14:  end if
15: end while
16: Estende  $Fin$  con tutti gli eventi contenuti in cut-off ed una
17: condizione  $(p, e)$  per ogni posto di output  $p$  di  $t$ ;
```

Si può notare come nell'algoritmo aggiornato, viene creato un insieme di eventi cut-off, che estenderanno il prefisso solo al termine dell'esecuzione.

La funzione UpdatePotExt

Come esposto in [6], uno dei motivi per cui la ricerca di possibili estensioni è molto onerosa a livello computazionale dipende dal dover aggiungere nel prefisso le condizioni *successive* ad un evento e preso in considerazione, ovvero le condizioni $c \in e^\bullet$, una alla volta. A causa di ciò, una transizione t , che consuma più di una condizione presente in $e^\bullet$, non può essere inserita all'interno del prefisso fino a quando tutte le condizioni ad essa precedenti non siano già state inserite. Questo porta ad un'ulteriore complicazione: la possibilità di inserire la transizione t all'interno del prefisso viene valutata ogni volta che una condizione consumata da t viene inserita, per potersi appunto accertare della possibilità di inserire la transizione. Nel caso di transizioni con preset grandi, il processo può diventare molto lento. Una possibile soluzione è quella di inserire nell'unfolding l'intero postset $e^\bullet$ in una volta sola [6].

Per poter fare questo, è importante riuscire a capire quando un possibile evento f successivo è separato da un evento e già inserito nell'unfolding, in modo da poter controllare, ed eventualmente inserire prima di f altre condizioni o eventi intermedi tra e ed f .

Eventi separati [6] Siano e ed f due eventi in un branching process di una rete di Petri, tali che $f \in (e^\bullet)^\bullet$ e $h(e^\bullet \cap \bullet f) \neq h(e)^\bullet \cap \bullet h(f)$. Allora e ed f sono separati [6].

Questa definizione suggerisce che tra e ed f possono esistere dei posti e delle transizioni intermedie, nonostante f rappresenti una transizione direttamente successiva al postset di e , ovvero $f \in (e^\bullet)^\bullet$.

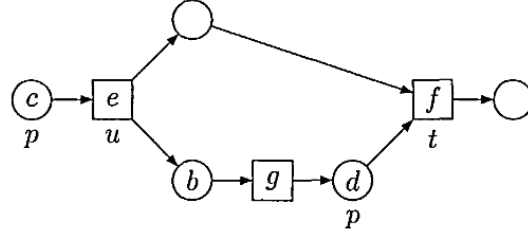


Figura 2.3: Una rete in cui e ed f risultano separati [6].

Nella figura 2.3 si può notare come gli eventi che rappresentano le transizioni e ed f siano separati: nonostante f faccia parte di $(e^\bullet)^\bullet$, non può essere inserito in un prefisso fin quando il ramo che rappresenta $b \xrightarrow{g} d$ non viene inserito nel prefisso. Se π risulta essere il branching process di una rete di Petri, e un evento di π e (t, D) un'estensione di tale evento, allora risulta che la dimensione dell'intersezione del postset di e con le condizioni D , precedenti a t risulta: $|e^\bullet \cap D| = |h(e^\bullet) \cap \bullet t|$ [6].

Basandosi sulle etichette utilizzate nella figura 2.3, un altro modo per limitare il numero di chiamate alla funzione PE , risulta nell'ignorare alcune delle transizioni in $(u^\bullet)^\bullet$ che l'algoritmo prova ad inserire dopo evento e etichettato da u . Se il preset $\bullet t$ di una transizione $t \in (u^\bullet)^\bullet$ ha una intersezione non vuota con $\bullet u \setminus u^\bullet$, ovvero $\bullet t$ condivide delle condizioni con $\bullet u$, allora t non può essere eseguita immediatamente dopo u , di conseguenza un'istanza f di t non può essere inserita nel prefisso subito dopo un evento e etichettato da u , anche se f potrebbe consumare delle condizioni prodotte da e [6]. Questo si può osservare nella figura 2.3: se $\bullet t \cap (\bullet u \setminus u^\bullet) \neq \emptyset$, allora l'evento f non può essere inserito nel prefisso, anche se consuma una condizione prodotta da e .

Espandendo la precedente definizione di eventi separati, se e ed f sono due eventi nell'unfolding di una rete di Petri tale che $f \in (e^\bullet)^\bullet$ e $\bullet h(f) \cap (\bullet h(e) \setminus h(e)^\bullet) \neq \emptyset$, allora e ed f sono separati [6]. Diventa quindi ovvio che se (t, D) è un'estensione di un evento e del branching process di una rete di Petri, allora $\bullet t \cap (\bullet h(e) \setminus h(e)^\bullet) = \emptyset$ [6].

Grazie a queste nuove definizioni, l'algoritmo potrebbe limitarsi a considerare solamente le transizioni facenti parte dell'insieme $(u^\bullet)^\bullet \setminus (\bullet u \setminus u^\bullet)^\bullet$ invece di considerare l'intero insieme $(u^\bullet)^\bullet$, ciò viene riportato nella funzione $UpdatePotExt(\pi, e)$ dell'algoritmo 5. Siccome è necessario trovare, in maniera efficiente, tutte le condizioni che risultano essere concorrenti ad una condizione d , quest'ultima facente parte del preset di t , si può usare la funzione $Cover(C, t, preset)$: segna le condizioni non concorrenti ad d come *inutilizzabili*, aggiunge d al preset, ancora incompleto di t , e successivamente aggiunge $(t, preset)$ all'insieme delle *estensioni* [6].

Di seguito le funzioni $UpdatePotExt(\pi, e)$ e $Cover(C, t, preset)$

Algoritmo 5 Le funzioni UpdatePotExt e Cover [6]

```

1: function UPDATEPOTEXT( $\pi, e$ )
2:    $extensions := \emptyset$ 
3:    $u := h(e)$ 
4:   for all  $t \in (u^\bullet)^\bullet \setminus (\bullet u \setminus u^\bullet)^\bullet$  do
5:      $preset := \{b \in e^\bullet \mid h(b) \in^\bullet t\}$ 
6:      $C :=$  tutte le condizioni di  $\pi$  concorrenti con  $e$ 
7:     Cover( $C, t, preset$ )
8:   end for
9:   return  $extensions$ 
10: end function
11:
12: function COVER( $C, t, preset$ )
13:   if  $|\cdot t| = |preset|$  then
14:      $extensions := extensions \cup \{(t, preset)\}$ 
15:   else
16:     sceglie  $p \in \bullet t \setminus h(preset)$ 
17:     for all  $d \in C$  tali che  $h(d) = p$  do
18:        $C' = \{c \in C \mid c \text{ co } d\}$ 
19:       Cover( $C', t, preset \cup \{d\}$ )
20:     end for
21:   end if
22: end function

```

Una casistica particolare si ha quando una transizione t presenta un *cappio*, o *self-loop*, ovvero quando $t \in (t^\bullet)^\bullet$. Ci sono tre casi che interessano questa tipologia di transizione [6]:

- $\bullet t \subset t^\bullet$. In questo caso t deve risultare inattiva, altrimenti la rete non risulta essere safe. L'esecuzione di t , produce tutti i token necessari per poter eseguire un'altra volta t . Se t non è inattiva, può essere eseguita un numero arbitrario di volte, producendo token per i posti ottenuti da $t^\bullet \setminus \bullet t \neq \emptyset$.
- $\bullet t = t^\bullet$. In questo caso, per ogni istanza e di t , già inserita nel prefisso, solamente un'istanza f di t può essere inserita direttamente dopo e , producendo un evento cut-off, in quanto $Mark[e] = Mark[f]$ e $[e] \subset [f]$, ovvero $e \prec f$ secondo un ordine adeguato $\prec$.
- $\bullet t \cap t^\bullet \neq \emptyset$ e $\bullet t \not\subseteq t^\bullet$. In questo caso, a seguito delle definizioni precedentemente descritte, un'altra istanza di t non può essere inserita direttamente dopo t .

Considerate queste casistiche, risulta che una nuova istanza di t può essere inserita dopo un altro evento già etichettato da t solo se $\bullet t = t^\bullet$ [6].

2.6 Utilizzi dei prefissi dell'unfolding di una rete di Petri

L'unfolding delle reti di Petri venne introdotto in [11] come possibile metodo per la risoluzione del problema dello *state explosion*, un problema ricorrente nei metodi di analisi e verifica del comportamento di sistemi concorrenti. In particolare, considerando un sistema concorrente, la dimensione dello spazio totale degli stati analizzabili cresce esponenzialmente rispetto al numero di processi e variabili del sistema stesso, nel caso delle reti di Petri rispetto al numero di posti e transizioni, rendendo qualsiasi metodo di analisi praticamente impossibile per reti di grandi dimensioni, in quanto un sistema potrebbe avere fino a m^n stati raggiungibili, con m il numero massimo di stati locali possibili per ogni componente della rete, ed n il numero di componenti della rete [12]. Il prefisso dell'unfolding di una rete di Petri, essendo una rete aciclica rappresentante tutti gli stati raggiungibili, riesce a mitigare il problema dello *state explosion*, risultando in un ottimo strumento per l'analisi della raggiungibilità degli stati e per l'analisi di situazioni di deadlock per una rete di Petri[11].

Raggiungibilità

Il problema della raggiungibilità viene definito in [13] come segue: dato un sottoinsieme A di posti di una rete, decidere se esiste una marcatura raggiungibile che marca tutti i posti di A .

In altre parole, analizzare la raggiungibilità di una rete consiste nel verificare se un determinato stato della rete viene raggiunto, e in che modo questo accade, dato che il prefisso dell'unfolding di una rete di Petri rende esplicite tutte le possibili esecuzioni della rete, incluse le relazioni di causalità e concorrenza tra gli eventi [14]. Ad esempio, chiedendosi se una determinata transizione t viene raggiunta in qualche esecuzione di una rete di Petri, si può esplorare il prefisso dell'unfolding fin quando un evento, che rappresenta la transizione t , non viene trovato o, al contrario, se la transizione t non viene eseguita affatto [14], oppure si può analizzare se, dato un insieme di transizioni, queste vengano eseguite più, o infinite, volte in determinate esecuzioni di una rete [14]. Grazie a questa proprietà, il prefisso dell'unfolding di una rete può essere utilizzato per scovare errori e bug in sistemi concorrenti, siano essi sistemi hardware o software, risultando utile per trovare, possibili deadlock nell'esecuzione del sistema.

In una rete di Petri, una marcatura che non abilita nessuna transizione è chiamata marcatura terminale, considerando che un unfolding rappresenta tutte le marcature raggiungibili di una rete, se le marcature terminali esistono, queste vengono rappresentate in esso. In termini più formali, in una rete non vi sono deadlock se ogni configurazione dell'unfolding può essere estesa verso una configurazione che contiene almeno un evento cut-off, ovvero, l'unfolding si può espandere infinitamente. In caso contrario è presente un deadlock [15].

Nella figura 2.4 si può osservare una rete con due possibili marcature terminali ed un prefisso del suo unfolding: l'esecuzione della transizione $t4$, porta ad uno stato di deadlock, così come l'esecuzione di $t2$ seguita da $t1$.

La rete è ottenuta da [15], modificata per ottenere delle marcature terminali.

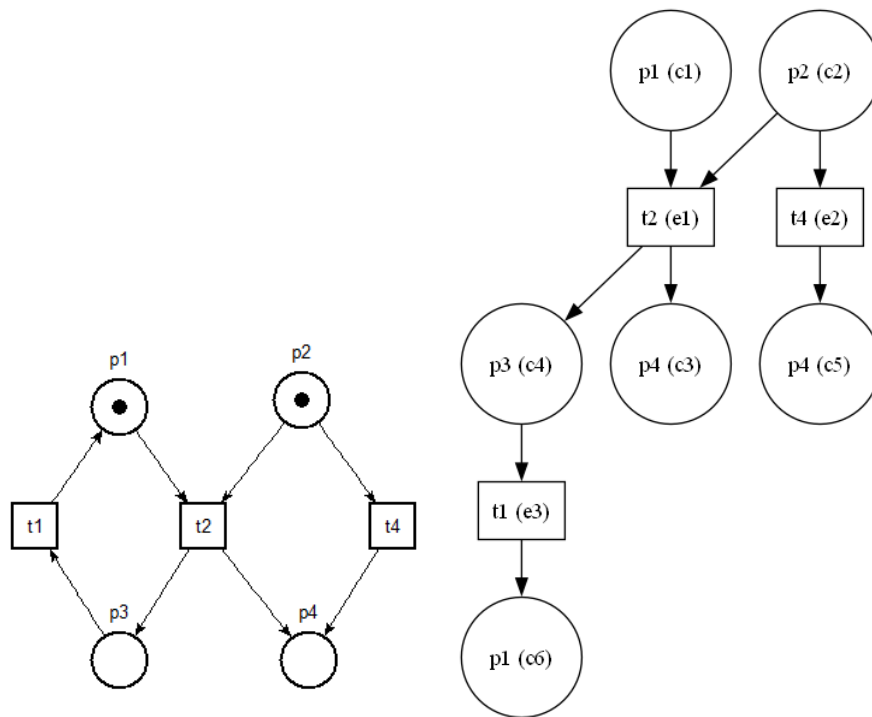


Figura 2.4: Una rete di Petri con solo marcature terminali e un suo unfolding

Si può notare la differenza con la rete originale rappresentata nella figura 2.5, in cui la transizione $t3$ non è stata omessa. Questa rete non genera un deadlock, è sempre presente almeno una transizione eseguibile che permette alla rete di non raggiungere marcature terminali.

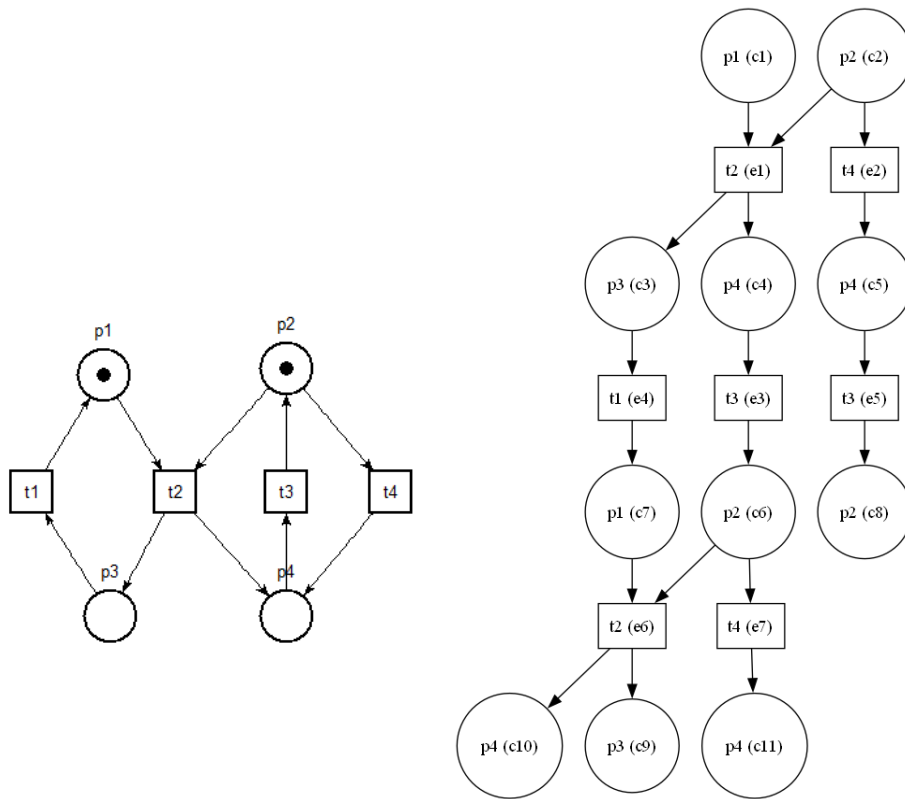


Figura 2.5: La rete originale senza deadlock [15] ed il prefisso del suo unfolding

Capitolo 3

Directed Unfolding di reti di Petri

Gli algoritmi per la generazione di prefissi dell'unfolding di una rete di Petri sono solitamente implementati con una strategia di ricerca breadth-first, sfruttando la lunghezza delle configurazioni della rete per trovare nodi da aggiungere al prefisso e per identificare eventi cut-off. Questa strategia può diventare molto inefficiente nel caso l'unfolding in costruzione diventi grande, rendendo le configurazioni più profonde e complesse da analizzare [4].

In [4] viene quindi proposta la tecnica del *Directed Unfolding*, una tecnica che sfrutta l'informazione riguardante una particolare marcatura cercata per guidare il processo di ricerca di nodi successivi. Siccome l'unfolding è principalmente usato per provare l'assenza di deadlock, gli algoritmi impiegati per generarne il prefisso si concentrano, solitamente, sul renderlo più piccolo, senza dedicare particolare attenzione al compito di renderne più efficiente il processo di ricerca di estensioni e di eventi cut-off [4].

In particolare si sfrutta l'informazione specifica della marcatura cercata sotto forma di una funzione euristica, utilizzata per stimare la distanza più breve da uno stato ad un altro, esplorando le scelte per estendere il prefisso in ordine crescente di distanza stimata. Per poter fare questo è stato definito un ordine semi-adequato [4], più debole di quello descritto in [3], considerato troppo restrittivo per il compito del directed unfolding. Risulta che, se l'euristica scelta è *ammissibile*, ovvero un'euristica che non sovrastima le distanze più brevi, allora il directed unfolding trova effettivamente la distanza più breve verso la marcatura cercata [4].

In [4] viene quindi fornita una definizione più formale del problema di raggiungibilità per reti di Petri 1-safe:

Raggiungibilità Data una rete di Petri $\Sigma = (P, T, W, M_0)$ e un sottoinsieme di posti $P' \subseteq P$, determina se esiste una sequenza di occorrenze σ tale che $M_0 \xrightarrow{\sigma} M$, dove $M(p) = 1$, per ogni posto $p \in P'$.

In altre parole, determina se esiste una sequenza di occorrenze di transizioni che dalla marcatura iniziale portano ad una marcatura in cui tutti i posti hanno un token. Per determinare la raggiungibilità di una rete di Petri vengono usati gli algoritmi di unfolding, ma, quando è ricercata la raggiungibilità di una sola

marcatura, il metodo definito *on-the-fly* è considerato più efficiente [4]: introduce una nuova transizione t_R nella rete originale, tale che il preset di questa transizione equivale al sottoinsieme P' di posti, ovvero $\bullet t_R = P'$. Si procede costruendo l'unfolding della rete, che viene fermato quando un evento e_R , che etichetta la transizione t_R , ovvero quando si ha che $\varphi(e_R) = t_R$, viene estratto dalla coda di possibili estensioni. Questo garantisce che l'insieme di posti P' è raggiungibile, in caso contrario viene creato l'intero prefisso finito completo, indicando che P' non è raggiungibile [4].

3.1 Dirigere l'unfolding

L'obiettivo del directed unfolding è quello di sfruttare l'informazione ottenuta dalla risoluzione del problema di raggiungibilità per dirigere il processo di costruzione del prefisso, ottenendo un algoritmo di unfolding il cui compito è la risoluzione del problema di raggiungibilità [4].

Per risolvere la raggiungibilità, l'unfolding può essere considerato come un processo di ricerca della transizione t_R , in modo che, nel momento della selezione di un nuovo evento dalla coda di possibili estensioni, vengano scelti quelli considerati più vicini alla transizione t_R , rendendone la ricerca più efficiente e giustificando il termine *directed*, inteso con il significato di "dirigere", che viene dato a questa strategia [4]. Questo processo è possibile solo se il prefisso è garantito essere completo, altrimenti l'algoritmo potrebbe fornire come risultato, sbagliando, la non raggiungibilità di t_R , per questo motivo l'ordine adeguato fornito in [3] viene sostituito da una sua definizione più debole [4].

Un ordine semi-adequato

Il nuovo ordine viene indicato in [4] con il simbolo $\prec_f$, e si basa su una funzione f che associa alle configurazioni dei numeri non negativi. La funzione f definisce gli ordinamenti $\prec_f$ come [4]:

$$C \prec_f C' \text{ se e solo se } \begin{cases} f(C) < f(C') & \text{se } f(C) < \infty \\ |C| < |C'| & \text{se } f(C) = f(C') = \infty \end{cases}$$

La funzione f si compone inoltre di due componenti: $f(C) = g(C) + h(C)$, dove:

- $g(C) = |C|$, ovvero $g(C)$ rappresenta il numero di eventi presenti nella configurazione C ;
- $h(C)$ è una funzione non negativa che vale $h(C) = 0$ se la transizione t_R è presente all'interno di C . Di conseguenza, per tutte le coppie di configurazioni C_1 e C_2 , tali che $\text{Mark}(C_1) = \text{Mark}(C_2)$ si ha che $h(C_1) = h(C_2)$.

La componente g è definita come il costo della configurazione C , h viene considerata la distanza per raggiungere la transizione t_R dalla marcatura di C [4]. Viene inoltre definita $h^*(C) = |C'| - |C|$, dove $C' \supseteq C$ è la configurazione di cardinalità minima che contiene t_R , se tale configurazione esiste, altrimenti vale ∞ . Si dice che h è un'euristica ammissibile se $h(C) < h^*(C)$ per tutte le configurazioni C , mentre una configurazione C^* è detta ottima se $t_R \in C^*$ e

C^* è la configurazione di cardinalità minima tra le configurazioni [4]. Viene quindi definito in [4] il seguente teorema, su cui si basa la tecnica del directed unfolding:

Teorema Sia $f(C) = g(C) + h(C)$ e si consideri l'ordinamento $\prec_f$. Allora:

1. L'algoritmo di unfolding, descritto in [3], accoppiato all'ordinamento $\prec_f$ risolve il problema di raggiungibilità;
2. Trova una configurazione ottima, se ne esiste una, e h è ammissibile.

La semi-adequatezza dell'ordine $\prec_f$ viene data sostituendo la seconda condizione che definisce un ordine parziale adeguato $\prec$ sulle configurazioni, definita di seguito: $C_1 \subset C_2$ implica che $C_1 \prec C_2$. Questa condizione garantisce la finitezza del prefisso, considerando n marcature raggiungibili della rete originale, e considerando una sequenza infinita di eventi $e_1 < e_2 < e_3 \dots$ nell'unfolding, si ha che vi sono $i < j \leq n+1$ marcature raggiungibili tali che $\text{Mark}(e_i) = \text{Mark}(e_j)$, e dato che $[e_i] \subset [e_j]$, la condizione precedentemente descritta implica che $[e_i] \prec [e_j]$ [4]. Si può definire una versione più debole di questo ordinamento:

Ordine semi-adequato [4] Un ordine parziale $\prec$ su configurazioni finite è detto *semi-adequato* se:

- $\prec$ è ben fondato
- In ogni catena infinita di configurazioni $C_1 \subset C_2 \subset C_3 \subset \dots$, ci sono $i < j$ tali che $C_i \prec C_j$
- $\prec$ viene mantenuto nel caso ci siano delle estensioni alle configurazioni: se $C_1 \prec C_2$ e $\text{Mark}(C_1) = \text{Mark}(C_2)$, per tutte le estensioni finite $C_1 \oplus E_1$ e $C_2 \oplus E_2$, con E_1 ed E_2 isomorfe, si ha che $C_1 \oplus E_1 \prec C_2 \oplus E_2$.

La seconda condizione viene quindi sostituita da una condizione più debole di quella proposta in [3].

3.2 La dimensione del prefisso finito

Il prefisso generato con il metodo del directed unfolding potrebbe risultare più piccolo, ed in alcuni casi addirittura incompleto, se confrontato con la tecnica dell'unfolding descritta nel capitolo 2. La funzione euristica h viene dotata di una ulteriore proprietà: h è in grado di fornire un criterio di eliminazione sicuro se e solo se $h(C) = \infty$ implica che non c'è nessuna configurazione $C' \supseteq C$ con $t_R \in C'$, ovvero $h^*(C) = \infty$ [4]. Questa proprietà implica che l'unfolding può essere fermato nel momento in cui il valore di f del miglior evento estratto dalla coda di eventi, è uguale a ∞ . Questo significa che la transizione t_R è non raggiungibile, trovando appunto un caso di deadlock. Come già anticipato, il prefisso generato con questa tecnica potrebbe risultare, non solo più piccolo di un prefisso generato con l'unfolding base, ma addirittura incompleto, in quanto mancherebbero tutte quelle marcature non necessarie al raggiungimento della transizione t_R [4], rese superflue dal fatto che l'interesse viene posto nel trovare una particolare transizione e non tutte le possibili esecuzioni di una rete di

Petri. Il prefisso generato è quindi sufficiente per decidere la raggiungibilità dell'obiettivo.

Se l'obiettivo non è raggiungibile, h non peggiora il comportamento dell'algoritmo di unfolding, ma anzi, lo migliora riducendo la dimensione del prefisso, come dimostrato dal seguente teorema, dove, per una funzione euristica h , $\text{ERV}(h)$ è l'algoritmo di unfolding direzionato dalla funzione h e $\beta(h)$ il prefisso generato da tale algoritmo:

Teorema [4] Sia $f^* = |C|$ il costo di una configurazione ottima C^* , e, nel caso una soluzione non esista, sia ∞ :

- Se h è ammissibile, allora tutti gli eventi $e \in \beta(h)$ hanno un valore di $f \leq f^*$, ovvero, nessun evento nel prefisso ha un valore peggiore del costo della configurazione ottima, non esplorando configurazioni con valore stimato peggiore.
- Se $h_1 \leq h_2$ sono due euristiche ammissibili, allora gli eventi $e \in \beta(h_2)$, con $g([e]) + h_2(e) < f^*$ sono anche in $\beta(h_1)$. Se inoltre, h_1 e h_2 sono entrambe monotone, e a parità di tie-breaking, allora $\beta(h_2) \subseteq \beta(h_1)$.

Diventa quindi naturale concludere che se h è ammissibile e t_R non è raggiungibile, il prefisso $\beta(h)$ è sempre più piccolo del prefisso generato da un algoritmo ERV classico, in quanto alcune possibili configurazioni vengono ignorate [4].

3.3 Euristiche

Per poter costruire delle funzioni euristiche, una metodologia comune risulta essere quella di definire un *rilassamento* del problema di ricerca, in modo che il problema rilassato possa essere risolto in maniera efficiente e, successivamente, utilizzarne il costo della soluzione come stima del costo della soluzione del problema reale [4]. Il problema di estendere una configurazione C dell'unfolding in una che includa anche la transizione obiettivo t_R corrisponde al problema della raggiungibilità della transizione t_R stessa, questo è il problema da rilassare per ottenere la stima della distanza per raggiungere t_R da C [4].

In [4] viene affermato che la sperimentazione è stata effettuata con due tipi di rilassamenti. Il primo tipo di rilassamento consiste nel considerare ogni posto nel preset di una transizione in maniera indipendente dalle altre. Una transizione t può scattare solamente se in ogni posto in $\bullet t$ è presente un token, di conseguenza la distanza stimata da una marcatura data M , verso una marcatura dove t può accadere, equivale a $d(M, \bullet t) = \max_{p \in \bullet t} d(M, \{p\})$, dove $d(M, \{p\})$ indica la distanza stimata da M ad una qualsiasi marcatura che includa $\{p\}$ [4]. Considerando che in un posto p per essere presente un token, almeno una transizione in $\bullet p$ deve occorrere, la stima precedentemente descritta risulta essere $d(M, \{p\}) = 1 + \min_{t \in \bullet p} d(M, \bullet t)$. Combinando queste due stime delle distanze, da [4] si ha che la distanza stimata, da una marcatura M a M' , può valere:

$$d(M, M') = \begin{cases} 0 & \text{se } M' \subseteq M \\ 1 + \min_{t \in \bullet p} d(M, \bullet t) & \text{se } M' = \{p\} \\ \max_{p \in M'} d(M, \{p\}) & \text{altrimenti} \end{cases}$$

Si ottiene quindi una funzione euristica $h^{\max}(C) = d(\text{Mark}(C), \bullet t_R)$, una funzione che stima una distanza mai più grande della distanza del problema non rilassato, quindi $h^{\max}$ è ammissibile [4].

Siccome le euristiche ammissibili non devono mai sovrastimare la stima della distanza verso l'obiettivo, possono risultare troppo conservative e potrebbero esse meno informative riguardo le distanze tra i diversi stati: un'euristica ammissibile potrebbe garantire la correttezza della soluzione del rilassamento, ma può risultare troppo debole per guidare effettivamente l'unfolding [4], soprattutto nel caso di configurazioni con molti conflitti e dipendenze. Un'euristica non ammissibile invece, risulta più libera nell'assegnare i valori di distanza alle varie configurazioni, fornendo più informazioni. Un'euristica inammissibile, ma più informativa di $h^{\max}$, è chiamata h^{sum} , ottenuta sostituendo $\max_{p \in M'}(M, \{p\})$ con la somma delle distanze per arrivare da una marcatura ad un posto p : $\sum_{p \in M'} d(M, \{p\})$ [4].

Ad esempio, se per raggiungere una transizione obiettivo sono disponibili due ramificazioni:

- Primo ramo C_1 : deve scattare una transizione con 5 posti diversi nel preset, ognuna delle quali dista 10 passi dalla marcatura attuale;
- Secondo ramo C_2 : deve scattare una transizione con solamente un posto nel preset, anch'esso distante 10 passi.

Per la funzione euristica $h^{\max}$ risulta che $h^{\max}(C_1) = 10$ ed anche $h^{\max}(C_2) = 10$ dato che questa funzione euristica considera solamente il valore di passi massimo, di conseguenza, entrambe le ramificazioni risulterebbero uguali, nonostante C_1 sia molto più complessa da esplorare rispetto a C_2 . Utilizzando h^{sum} , e quindi sommando tutte le distanze effettive necessarie per raggiungere un posto, risulta che $h^{\text{sum}}(C_1) = 50$, ovvero 10 passi per ogni posto nel preset della transizione, mentre $h^{\text{sum}}(C_2) = 10$, in quanto in questa configurazione è presente un solo "percorso" di distanza 10, risultando chiaramente la scelta migliore per dirigere l'unfolding.

Il secondo tipo di rilassamento viene chiamato *delete relaxation* [4]. In questo tipo di rilassamento, si assume che una transizione richieda solamente un token in ogni posto nel suo preset, ma non li consuma quando occorre. Di conseguenza, una volta marcato, un posto non perde mai il suo token. In questo modo ogni marcatura raggiungibile nella rete originale, è un sottoinsieme di una marcatura raggiungibile nel sistema rilassato [4]. Il problema delete-rilassato ha la proprietà che, se una soluzione esiste, può essere trovata in tempo polinomiale [4]. La procedura per fare ciò genera una occurrence net, detta *relaxed plan graph* [4], ovvero un prefisso completo dell'unfolding del problema rilassato. Grazie a questo rilassamento, il relaxed plan graph è un prefisso più semplice del prefisso dell'unfolding di una rete di Petri, risultando essere senza conflitti e con transizioni che possono comparire al massimo una volta sola.

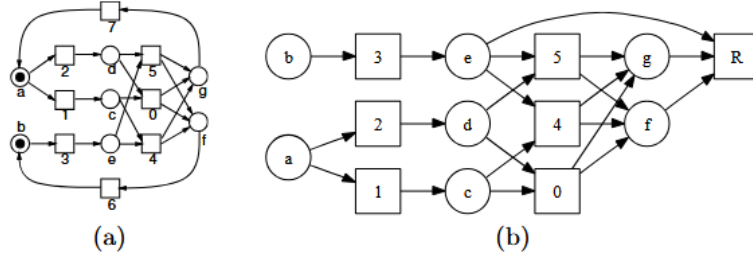


Figura 3.1: Una rete di Petri (a) e il suo relaxed plan graph (b) [4]

A seguito della costruzione di questo grafo, si può controllare se la transizione t_R è raggiungibile, e la configurazione che porta a t_R viene considerata la soluzione del problema rilassato, quest'ultima viene scelta in modo arbitrario se sono presenti più configurazioni [4]. La dimensione di questa soluzione fornisce il valore della funzione euristica, chiamata in questo caso h^{FF} [4].

Nella figura 3.1 si può vedere un esempio di relaxed plan graph, con la transizione R indicata come obiettivo. Una soluzione minima è la sequenza 3, 5, R , ma sono presenti altre soluzioni come 1, 3, 4, R oppure 1, 2, 0, 3, R [4].

Dato che una soluzione del grafo viene estratta arbitrariamente, questa euristica non è ammissibile: potrebbe sovrastimare o sottostimare la stima rispetto al problema reale, ma, siccome il calcolo della soluzione minima ammissibile, chiamata h^+ , è un problema NP-hard [4], si sacrifica l'ottimalità della soluzione per ottenere una funzione euristica più veloce.

Capitolo 4

Implementazione di un programma per la costruzione di prefissi finiti

Lo studio dell'argomento riguardante la generazione di prefissi dell'unfolding di una rete di Petri è stato necessario per lo svolgimento dello stage, tenuto in sede universitaria, avente come obiettivo la realizzazione di un prototipo software per la generazione di prefissi dell'unfolding. In quanto tale progetto di stage risulti essere un prototipo per poter generare prefissi di unfolding, le prestazioni computazionali sono state poste in secondo piano rispetto alla costruzione, e alla correttezza, del prefisso dell'unfolding.

4.1 Specifiche tecniche

L'applicativo è stato scritto in Python, linguaggio scelto per la semplicità sintattica e per la versatilità con cui le strutture dati, quali set o liste, possono essere utilizzate. Si è scelto di sacrificare possibili prestazioni computazionali migliori ottenibili con altri linguaggi, preferendo appunto la versatilità fornita da Python. Per tenere traccia delle varie funzionalità aggiunte durante lo sviluppo, è stato utilizzato *Git*, con il supporto aggiuntivo della piattaforma online *GitHub*.

Il programma è in grado di ottenere come input una rete di Petri costruita tramite il software **TINA** (TIme Petri Net Analyzer)¹ una toolbox in grado di costruire ed analizzare, in particolare, le Time Petri Nets ma perfettamente in grado di produrre reti di Petri ordinarie. Per lo scopo dello stage, il software TINA è stato utilizzato per disegnare ed editare le sole reti di Petri P/T, con particolare attenzione verso le reti 1-safe.

¹<https://projects.laas.fr/tina/index.php>

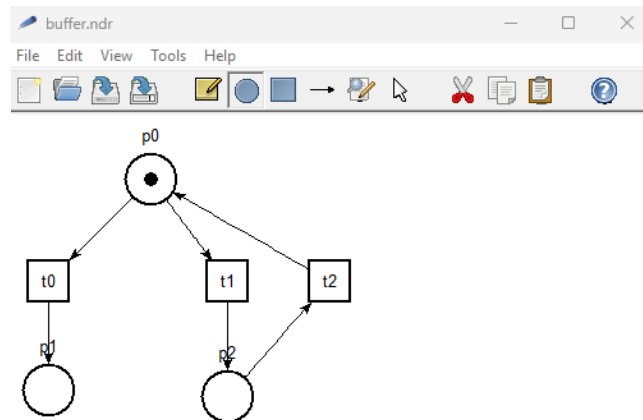


Figura 4.1: Toolbox TINA

Il software fornisce un file di testo **.ndr**, un formato testuale nativo di TINA per rappresentare reti di Petri, successivamente fornito all'unfolder sviluppato come argomento tramite una riga di comando da terminale:

```
python main.py <nome_rete>.ndr
```

Viene avviato in automatico il processo di costruzione del prefisso dell'unfolding, fornendo come output un file **.dot** avente lo stesso nome della rete utilizzata come argomento, visualizzabile tramite un qualsiasi editor online per file **.dot**, come ad esempio **edotor**², oppure, previa l'installazione del programma di visualizzazione di grafi **Graphviz**³, convertito in formato png tramite la seguente riga di comando da terminale:

```
dot -Tpng nome_rete.dot -o nome_rete.png
```

Dove *Tpng* significa semplicemente *To png*. Non è necessario mantenere il nome originale della rete in questo caso.

²<https://edotor.net/>

³<https://graphviz.org/>

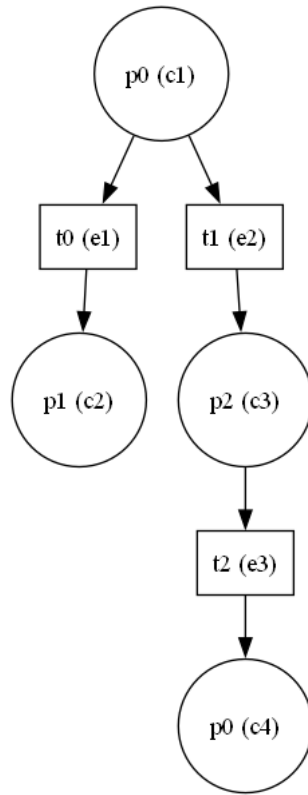


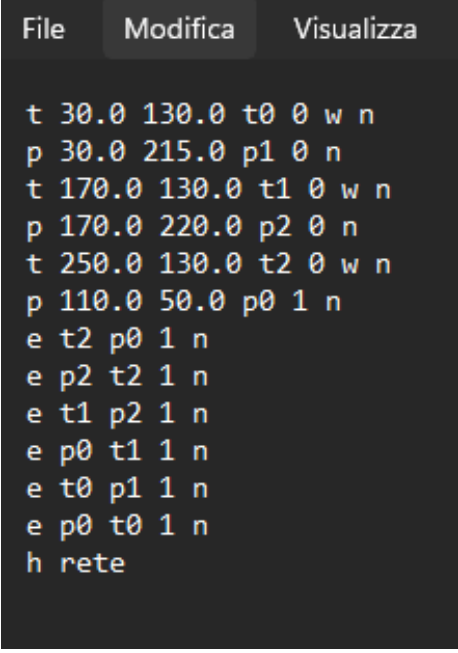
Figura 4.2: Prefisso di unfolding della rete presente nella figura 4.1, ottenuto tramite il software prodotto durante lo svolgimento dello stage

Il programma è composto da 4 moduli principali:

- **main.py**: il modulo main del programma, il cui compito è quello di chiamare le funzioni che generano la rete di Petri ed il prefisso del suo unfolding, oltre a chiamare la funzione che genera il file .dot finale;
- **branching_process.py**: è il modulo principale, qui viene creato il prefisso dell'unfolding, sono presenti tutte le strutture dati più significative del software, le classi *Event*, *Condition* e *Processor*, tutte le funzioni di generazione del prefisso oltre che la funzione che converte il prefisso in file .dot;
- **net.py**: modulo che permette di generare la rete di Petri. Sono presenti le classi *Place*, *Transition*, *Arc* e *PetriNet*, aventi il compito di rappresentare logicamente la rete di Petri originale;
- **parser.py**: il modulo che legge il file di TINA .ndr e chiama le funzioni per generare la rete di Petri.

4.2 Parser di reti di Petri

Come precedentemente anticipato, le reti di Petri utilizzare dal software sono disegnate tramite la toolbox TINA. Questo programma fornisce dei file con estensione .ndr. Sono dei file di testo che si riferiscono alla rappresentazione grafica della rete.



```
File Modifica Visualizza

t 30.0 130.0 t0 0 w n
p 30.0 215.0 p1 0 n
t 170.0 130.0 t1 0 w n
p 170.0 220.0 p2 0 n
t 250.0 130.0 t2 0 w n
p 110.0 50.0 p0 1 n
e t2 p0 1 n
e p2 t2 1 n
e t1 p2 1 n
e p0 t1 1 n
e t0 p1 1 n
e p0 t0 1 n
h rete
```

Figura 4.3: Un file che rappresenta una rete di Petri

Ogni elemento della rete viene identificato tramite una lettera iniziale:

- p per identificare i posti
- t per identificare le transizioni
- e (*edge*) per indicare gli archi

In ogni riga sono inseriti anche il nome del componente e, per i posti, il numero di token presenti nel momento in cui la rete viene disegnata. Per gli archi sono presenti il posto, o la transizione, di partenza e di destinazione, oltre che l'eventuale peso dell'arco, indicante il numero di token che tale arco consuma in caso di scatto di una transizione. Il software è in grado di generare reti con pesi > 1 , ma questa implementazione software è stata pensata con reti aventi tutti gli archi di peso 1. Altri elementi, come le coordinate del componente nell'area di disegno di TINA vengono ignorati.

Il file di testo, opportunamente fornito al programma come argomento in riga di comando, viene aperto da python tramite i classici metodi di lettura/scrittura dei file, già presenti nel linguaggio. Il parser verrà utilizzato per creare la rete vera e propria, per questo verrà chiamato nel file main. La funzione `parse()` si occuperà di leggere e processare il file.

Di seguito il principale caso d'uso della funzionalità del parser, il cui compito è fornire la rete.

```
1 net = PetriNet()
2 parser = Parser(filepath)
3
4 net = parser.parse()
5
```

Ogni riga viene letta all'interno di un ciclo for, e, all'inizio di ogni iterazione, viene divisa in colonne e considerata come un array tramite la seguente riga di codice:

```
1 v = line.split()
2
```

Le righe, essendo formate da componenti con posizioni precise, permettono una facile identificazione dei componenti da ignorare e quali invece utilizzare per creare un oggetto della rete. La prima lettera di ogni riga viene utilizzata per controllare che tipo di elemento della rete sta per essere processato, dopo questo controllo viene chiamata l'apposita funzione di creazione dell'elemento. Queste ultime verranno descritte più approfonditamente nella prossima sezione. Gli elementi non utilizzati, possono essere ignorati tramite il carattere `_`, il cui compito è appunto quello di ignorare dei valori quando questi hanno una precisa posizione all'interno di un array. Di seguito una parte del processo di creazione di un oggetto della rete, in particolare di un posto. Il processo è del tutto simile anche per gli altri elementi della rete:

```
1 if v[0] == "p":
2     if len(v) != 6:
3         raise ValueError("ERRORE: formato place non valido")
4
5     _, _, _, name, tokens, _ = v
6     net.addPlace(name, int(tokens))
7
```

Siccome ogni riga ha un numero di elementi preciso per ogni componente della rete, è possibile svolgere un controllo sulla validità di tale riga, semplicemente controllandone la lunghezza.

Nella figura 4.3 è possibile notare come ogni file finisca con la lettera *h* ed il nome della rete. Questa ultima componente è impiegata per terminare la lettura del file. La rete costruita verrà infine ritornata e il processo di parsing termina.

4.3 Il modulo `net.py` e la rete di Petri

Per poter correttamente rappresentare una rete di Petri, nel modulo `net.py` vengono impiegate 4 classi, ovvero `Place`, `Transition`, `Arc` e `PetriNet`.

In particolare, una rete di Petri è un oggetto composto da *dizionari* di oggetti di tipo `Place` e `Transition` oltre che ad una lista di archi, il cui compito è quello di collegare logicamente posti e transizioni.

Ogni posto ha un attributo *name* e un attributo *token*, quest'ultimo indicante il numero di token effettivamente presenti nel posto. Ogni transizione, invece, ha solo un attributo nome. Il collegamento tra posto e transizione avviene tramite gli archi, che uniscono logicamente le componenti. Un arco presenta un attributo source (**src**) e un attributo destination (**dst**), utilizzati per indicare il posto, o la transizione, di partenza e la rispettiva destinazione.

Per poter aggiungere ogni componente nella rete, esistono delle apposite funzioni all'interno della classe *PetriNet*: *addPlace*, *addTransition* e *addArc*, il cui compito è inserire in ogni struttura dati un oggetto del tipo corrispondente, chiamando il costruttore della classe di riferimento.

```

1  def addPlace(self, p_name, token):
2      self.places[p_name] = Place(p_name, token)
3

```

Il precedente è un esempio di come i posti vengono aggiunti alla rete di Petri. Si può notare come il nome del posto viene utilizzato anche come **key** per potersi riferire all'elemento con semplicità. Il processo d'inserimento è del tutto simile per le transizioni. L'aggiunta degli archi prevede un passaggio ulteriore, ovvero controllare se gli elementi *source* e *destination* siano già presenti nelle strutture dati della rete. Oltre a garantire la certezza che gli archi stiano collegando elementi esistenti, questo controllo serve anche a definire se l'arco va da un posto ad una transizione e viceversa. Di seguito la funzione *addArc()*, dove viene reso esplicito il controllo su posti e transizioni:

```

1  def addArcs(self, src, dst, weight=1):
2      if src in self.places and dst in self.transitions:
3          self.arcs.append(Arc(self.places[src], self.transitions
4                               [dst], weight))
5      elif src in self.transitions and dst in self.places:
6          self.arcs.append(Arc(self.transitions[src], self.places
7                               [dst], weight))
8      else:
9          raise ValueError("Errore: arco non valido")

```

A causa della natura di questo controllo, ne consegue che gli archi, nel momento in cui la rete viene disegnata all'interno di TINA, debbano essere inseriti successivamente a posti e transizioni, in quanto ogni elemento della rete viene salvato nel file di testo *.ndr* seguendo l'ordine d'inserimento nel disegno vero e proprio.

4.4 Il modulo *branching_process.py* e l'unfolding della rete di Petri

Il modulo *branching_process.py* contiene tutti gli elementi principali per costruire il prefisso dell'unfolding della rete di Petri.

Viene implementato l'algoritmo di unfolding, e la funzione per trovare possibili estensioni, presentati in [6], ed utilizza l'ordine parziale $\prec$ definito in [3]. Sono inoltre presenti le funzioni per gestire le relazioni di conflitto e causalità tra eventi, i quali vengono utilizzati per gestire anche la concorrenza.

Le classi *Event* e *Condition*

Eventi e condizioni sono rappresentati tramite due classi, *Event* e *Condition*, entrambe con attributi replicanti le caratteristiche di eventi e condizioni descritte in [3]. In particolare:

- La classe *Condition* presenta 3 attributi principali: un attributo *place*, ovvero l'oggetto di tipo posto a cui la condizione fa riferimento, un attributo *input_event* rappresentate l'evento che genera tale condizione e un id. L'id

si ottiene tramite una variabile `_counter` posta all'esterno del costruttore della classe. Ogni volta che una nuova condizione è creata, il contatore viene incrementato di 1. Di seguito il costruttore della classe `Condition` ed il suo contatore:

```

1      class Condition:
2          _counter = 1
3          def __init__(self, place, input_event=None):
4              self.place = place
5              self.event = input_event
6              self.id = Condition._counter
7              Condition._counter += 1
8

```

- La classe `Event` ha un attributo *transition* che si riferisce all'oggetto di tipo transizione rappresentato dall'evento `e`, siccome un evento può avere più di una condizione in input, un set di *input_conditions*. Come per la classe `Condition`, anche qui è presente un contatore utilizzato come id dell'evento. Inoltre, è presente un set di *output_conditions*: anche se non strettamente necessario, è un attributo molto utile in caso di visite dell'unfolding, come ad esempio in caso di stampe del prefisso.

```

1      class Event:
2          _counter = 1
3          def __init__(self, transition, input_conditions):
4              self.transition = transition
5              self.input_conditions = set(input_conditions)
6              self.output_conditions = set()
7
8              self.id = Event._counter
9              Event._counter += 1
10

```

La classe *Processor*

In questa classe sono presenti tutte le strutture dati e le funzioni che permettono il funzionamento del programma.

Nel costruttore della classe vengono inizializzate le strutture dati fondamentali per mantenere le relazioni tra eventi e condizioni, oltre che l'unfolding stesso. Per gli elementi propri dell'unfolding la scelta sul tipo di struttura dati è ricaduta sul `set`. Questo tipo di insieme, oltre a presentare numerose operazioni insiemistiche integrate, non permette la presenza di duplicati, una proprietà fondamentale per l'unfolding: anche se un elemento della rete di Petri può comparire più volte in un suo unfolding, eventi e condizioni vengono sempre considerati come elementi singoli e differenti.

```

1      def __init__(self, net):
2
3          self.net = net
4          self.min = set()
5
6          self.events = set()
7          self.conditions = set()
8
9          self.conflict_relation = set()
10         self.causal_relation_pre = set()
11         self.causal_relation_post = set()

```

```

12         self.cut_off = set()
13
14         self.min_by_marking = {}
15         self.config_cache = {}
16         self.config_length_cache = {}
17
18

```

- Il **set** denominato *min* è l'insieme delle condizioni della marcatura iniziale. La ricerca di tali condizioni è un processo molto semplice, si tratta di ricercare, all'interno dell'insieme dei posti della rete di Petri, i posti che presentano almeno un token al loro interno. Per ogni posto di questo tipo viene creata una condizione con valore `input_event` nullo.

I set *events* e *conditions* sono insiemi di comodo, utilizzati in caso di stampe o ricerche specifiche tra gli elementi di questi insiemi.

- Le relazioni di conflitto e causalità sono mantenute tramite dei set di *copie*, o Tuple. Quando due elementi dell'unfolding sono in conflitto, o in relazione causale, sono inseriti all'interno del relativo insieme come coppia, in maniera da poter controllare agilmente la loro relazione. Ogni volta che un elemento viene inserito nel prefisso, un'apposita funzione si occuperà di aggiornare queste relazioni, controllando l'ereditarietà delle stesse. In particolare, la relazione causale è divisa in due insiemi per questioni di praticità: l'insieme *causal_relation_pre* rappresenta la relazione causale da condizione ad evento, viceversa per l'insieme *causal_relation_post*. Questi tre insiemi permettono un controllo della relazioni di concorrenza *al volo* tramite una funzione, rendendo superflua la presenza di un ulteriore insieme che mantenga anche tale relazione.

- Il dizionario *min_by_marking* viene utilizzato nell'identificazione degli eventi cut-off, il compito di tale costrutto è quello di immagazzinare l'occorrenza minima di una marcatura in modo da poter controllare correttamente le condizioni di *troncamento* poste dall'ordine adeguato di [3].

I dizionari *config_cache* e *config_length_cache* sono impiegati nella ricerca delle configurazioni locali di un evento e nel calcolo della loro lunghezza, in modo da non dover eseguire più volte tali operazioni per uno stesso evento.

L'algoritmo di Unfolding

L'algoritmo implementato realizza la costruzione di un prefisso finito dell'unfolding della rete di Petri fornita come input, seguendo una versione lievemente modificata dell'algoritmo migliorato proposto in [6]. L'algoritmo genera, in maniera progressiva, eventi e condizioni, aggiornando, durante la sua esecuzione, le relazioni di conflitto e causalità, individuando eventi di tipo cut-off secondo l'ordine adeguato presentato in [3].

L'algoritmo ha inizio con la ricerca delle condizioni corrispondenti alla marcatura iniziale della rete. Tali condizioni costituiscono l'insieme di partenza del prefisso, denotato con un set di nome *unf*. Ad ogni iterazione, l'insieme *unf* rappresenta un prefisso, in costruzione, dell'unfolding della rete iniziale. Vengono quindi ricercate le prime estensioni tramite la funzione *initial_extensions()*, ovvero vengono identificate le transizioni abilitate dalla marcatura iniziale.

La funzione `initial_extensions()` riceve come input l'unfolding *unf* iniziale, e fornisce in output un insieme di eventi. Come prima operazione, estrae le condizioni della marcatura iniziale presenti in *unf*. Per ogni transizione *t* presente nella rete di Petri originale, calcola il suo preset, individua le condizioni della marcatura iniziale corrispondenti ai posti presenti nel preset, inserendoli in un opportuno insieme:

```

1   in_conditions = {c for c in initial_markings if c.place in
2   preset_places}

```

Successivamente viene effettuato un controllo per appurare che tutte le condizioni richieste siano presenti, comparando la lunghezza degli insiemi delle condizioni e del preset, se tale controllo è affermativo, viene generato un nuovo evento:

```

1   new_event = Event(t, in_conditions)
2

```

Per ciascun posto del postset di *t*, viene creata una condizione di output associata al nuovo evento. L'insieme di eventi risultante costituisce le prime possibili estensioni del prefisso.

L'algoritmo procede iterando sull'insieme delle possibili estensioni finché queste esistono. Il primo passo consiste nel determinare l'evento *minimo* tra le possibili estensioni. Tale evento, denominato *candidate_event* viene aggiunto all'insieme *unf*, insieme ad ogni condizione di output associata all'evento. Vengono aggiornate le relazioni di causalità e conflitto tra le nuove componenti dell'unfolding e infine viene verificata la proprietà di cut-off. Se l'evento non è di tipo cut-off, vengono calcolate le possibili estensioni, a partire dall'evento candidato, e vengono aggiunte all'insieme delle possibili estensioni. In caso contrario, l'evento viene considerato cut-off, venendo aggiunto all'interno dell'insieme apposito. In questa casistica, l'insieme delle possibili estensioni non viene espanso, dando la possibilità al ciclo di terminare nel momento in cui tale insieme rimane vuoto. Infine viene ritornato l'insieme *unf*, ponendo fine alla generazione del prefisso.

Trovare possibili estensioni

La ricerca di possibili estensioni è il processo più oneroso, a livello computazionale, nella costruzione di prefissi completi di unfolding. Per implementare questa operazione sono state impiegate le funzioni *update_pot_ext()* e *cover()* seguendo l'approccio descritto in [6], il cui obiettivo è rendere più efficiente la ricerca di possibili estensioni, impiegando delle euristiche sulle transizioni della rete di Petri.

La funzione *update_pot_ext()* ha il compito di calcolare l'insieme delle possibili estensioni dell'unfolding, a partire dall'ultimo evento, *e*, inserito in esso. Viene *estratta* la transizione *u* etichettata dall'evento impiegato e vengono considerate solo le transizioni, raggiungibili dal postset di *u*, che appartengono all'insieme $(u^*)^* \setminus (*u \setminus u^*)^*$. Questo insieme esclude le transizioni dipendenti da posti presenti nel preset di *u* ma che non influenzano l'evento *e*, come riportato in [6]. Per ogni transizione *t* candidata viene calcolato il suo preset e un insieme *C* di condizioni, tali che appartengano a tale preset e che siano concorrenti con l'evento *e* considerato. Tale insieme di condizioni viene passato alla funzione *cover()*, il cui compito è quello di costruire, ricorsivamente, il preset di un nuovo evento, a partire dall'insieme *C*.

- Caso base: il preset risulta completo, viene effettuato un controllo sulle condizioni, per verificare che esse siano mutualmente concorrenti. Viene controllato che non esista un evento con la stessa transizione e lo stesso insieme di condizioni di input, per evitare duplicati. Questo controllo è necessario in quanto un evento viene generato solamente nel caso non esista un altro evento equivalente nell'unfolding. Successivamente viene creato un nuovo evento, con gli stessi metodi descritti nel caso delle estensioni iniziali.
- Passo ricorsivo: viene selezionato un posto non ancora presente nell'insieme dei preset, successivamente vengono considerate tutte le condizioni associate a tale posto, generando un nuovo insieme di condizioni C , concorrenti alla condizione considerata. Infine la funzione richiama sé stessa, fino al completamento della costruzione del preset.

```

1  def cover(self, C, t, preset):
2      pre_t = self.find_preset(t)
3      extensions = set()
4
5      if len(pre_t) == len(preset): # caso base
6          in_conditions = {preset[p] for p in pre_t}
7          condition_list = list(in_conditions)
8          for i in range(len(condition_list)):
9              for j in range(i + 1, len(condition_list)):
10                 if not self.is_concurrent(condition_list[i],
condition_list[j]):
11                     return set()
12
13                 existing = self.existing_event(t, in_conditions)
14                 if existing:
15                     return set()
16
17                 new_event = Event(t, in_conditions)
18                 postset_places = {arc.dst for arc in self.net.arcs if
arc.src == t}
19                 for place in postset_places:
20                     new_condition = Condition(place, new_event)
21                     new_event.output_conditions.add(new_condition)
22
23                 extensions.add(new_event)
24                 return extensions
25             else: # passo ricorsivo
26
27                 diff = pre_t - set(preset.keys())
28                 p = next(iter(diff))
29
30                 for d in [cond for cond in C if cond.place == p]:
31                     C_first = {c for c in C if self.is_concurrent(c, d)}
32
33                 union_preset = dict(preset)
34                 union_preset[p] = d
35                 extensions |= self.cover(C_first, t, union_preset)
36                 return extensions

```

Codice 4.1: La funzione Cover()

4.5 Relazioni di conflitto, causalità e concorrenza

Per poter svolgere un corretto unfolding, è necessario mantenere le relazioni di conflitto, causalità concorrenza tra eventi e condizioni. Nell'implementazione software si è scelto di mantenere esplicite le relazioni di causalità e conflitto, tramite insiemi di coppie di eventi e condizioni, per poi derivare la relazione di concorrenza con dei semplici controlli, senza la necessità di rendere esplicita anche questa relazione, grazie alla definizione stessa di concorrenza: due elementi dell'unfolding sono concorrenti quando non sono né causali né in conflitto tra di loro.

Come già anticipato nell'introduzione della classe `Processor`, le relazioni di causalità e conflitto sono mantenute da strutture dati di tipo set:

- `causal_relation_pre`, rappresenta la relazione condizione→evento
- `causal_relation_post`, rappresenta la relazione evento→condizione
- `conflict_relation`, rappresenta il conflitto tra eventi

Causalità

La relazione di causalità viene aggiornata nell'algoritmo principale, ogni volta che un nuovo elemento viene inserito nell'insieme dell'unfolding. In particolare:

- Per ogni condizione di input di un evento, viene chiamata la funzione `causal_updater_pre()`, il cui compito è aggiungere la coppia (condizione, evento) nell'insieme apposito
- Per ogni condizione di output di un evento, viene chiamata la funzione `causal_updater_post()`, che aggiunge la relazione (evento, condizione).

Per verificare l'esistenza di una relazione di causalità tra due elementi dell'unfolding viene utilizzato un algoritmo di visita in ampiezza (BFS), tramite la funzione sul grafo di causalità `is_causal()`, la quale richiede in ingresso due attributi, sia eventi che condizioni. La ricerca viene svolta sugli eventi, per questo motivo, nel caso gli argomenti forniti risultino essere condizioni, vengono considerati gli eventi che le generano.

Nel caso le condizioni risultino essere quelle iniziali il controllo avrà esito nullo, in quanto due condizioni iniziali non appartengono a nessuna catena causale, se non a quelle di altri elementi del prefisso. Se due elementi corrispondono allo stesso evento, la causalità viene esclusa a priori.

La funzione inizializza una struttura di visita, denominata *cone*, contenente l'evento corrente *x*, e itera sulla stessa fino all'esaurimento di elementi al suo interno. Finché la struttura *cone* contiene elementi, estrae il primo evento corrente: se esso coincide con l'evento *y* ricercato, allora esiste una relazione causale, la funzione ritorna *true*. Altrimenti calcola le condizioni prodotte dall'evento, le quali verranno utilizzate per trovarne gli eventi successivi, inserendoli poi nella struttura di visita. Per evitare ridondanze durante la ricerca viene costruito un insieme *visited*, ovvero una struttura dati di tipo set contenente i nodi già esplorati. Nel caso il ciclo dovesse terminare senza esito, ovvero l'evento *y* non è stato trovato e la struttura di visita è vuota, non esiste causalità tra gli eventi *x* e *y*.

```

1  def is_causal(self, x, y):
2      x_ev = x.event if isinstance(x, Condition) else x
3      y_ev = y.event if isinstance(y, Condition) else y
4
5      if x_ev is None or y_ev is None:
6          return False
7
8      if x_ev is y_ev:
9          return False
10
11     visited = set()
12     cone = [x_ev]
13
14     while cone:
15         node = cone.pop()
16         if node == y_ev:
17             return True
18
19         if node in visited:
20             continue
21         visited.add(node)
22
23         produced_conditions = {cond for (e, cond) in self.
causal_relation_post if e is node}
24         successor_events = {ev2 for (cond2, ev2) in self.
causal_relation_pre if cond2 in produced_conditions}
25
26         cone.extend(successor_events)
27
28     return False
29

```

Codice 4.2: La funzione is_causal()

Conflitto

La relazione di conflitto è aggiunta, tramite la funzione *update_conflict_relation()*, confrontando il nuovo evento con tutti gli eventi già inseriti nel prefisso, semplicemente indicati come *e*. Si distinguono due casi principali in cui un conflitto può verificarsi:

- Conflitto diretto: due eventi competono per l'acquisizione di una stessa risorsa, ovvero condividono almeno una condizioni di input. In questo caso, la coppia rappresentante il conflitto, (evento, *e*), viene aggiunta nell'insieme delle relazioni di conflitto.
- Conflitto ereditato: l'evento è in conflitto con un evento appartenente alla storia causale delle sue condizioni di input. Se un evento e_1 è in conflitto con un evento e_2 , allora tutti gli eventi causalmente dipendenti da e_1 saranno in conflitto con e_2 , dato che l'esecuzione di un evento discendente da un altro evento, richiede l'esecuzione degli eventi precedenti ad esso. Per poter fare ciò, la funzione *update_conflict_relation()* prende in considerazione ogni condizione di input del nuovo evento, e controlla se l'evento che l'ha prodotta sia in conflitto con l'evento *e*. In caso affermativo, allora il nuovo evento è anche in conflitto con *e*.

```
1 def update_conflict_relation(self, event):
2     for e in self.events:
3         if e is event:
4             continue
5
6         if (event, e) in self.conflict_relation or (e, event)
7         in self.conflict_relation:
8             continue
9
10        if not event.input_conditions.isdisjoint(e.
11        input_conditions):
12            self.add_conflict(event, e)
13
14        for c in event.input_conditions:
15            if c.event and ((c.event, e) in self.
16            conflict_relation or (e, c.event) in self.conflict_relation):
17                self.add_conflict(event, e)
18                break
```

Codice 4.3: La funzione *update_conflict_relation()*

Concorrenza

Considerando che la relazione di concorrenza tra due elementi dell'unfolding risulta essere la conseguenza dell'assenza di causalità e conflitto tra di essi, è possibile calcolare questa relazione dinamicamente, tramite la funzione *is_concurrent()*, riducendo l'uso della memoria e sfruttando le informazioni già ottenute su causalità e conflitto. Essa può operare sia su eventi che su condizioni, restituendo un risultato booleano. Come nel caso riguardante il controllo di causalità tra due elementi, se uno di essi risulta essere una condizione, viene estratto l'evento che l'ha generata.

Nel caso entrambe le condizioni dovessero appartenere alla marcatura iniziale, ovvero il loro evento generatore è nullo, esse risultano sempre concorrenti in quanto non esistono dipendenze causali, o di conflitto, tra di esse.

Se uno dei due eventi risulta essere nullo, ovvero almeno uno dei due elementi appartiene alla marcatura iniziale, bisogna controllare se tale condizione non appartenga Né al preset dell'altro evento né nel preset di qualche suo antenato causale. Per questo motivo viene calcolata la configurazione locale dell'evento non iniziale, in modo da ottenere l'insieme di eventi che portano alla sua generazione. Per ogni evento nella configurazione locale appena calcolata, controlla se l'evento iniziale non faccia parte del preset. In caso affermativo la concorrenza non è presente ed il controllo termina.

```

1      if e1 is None and e2 is None:
2          return True
3
4      if e1 is None or e2 is None:
5          if e1 is None:
6              if n1 in e2.input_conditions:
7                  return False
8              config_e2 = self.local_configuration(e2)
9              for ancestor_event in config_e2:
10                 if n1 in ancestor_event.input_conditions:
11                     return False
12             return True
13         else:
14             if n2 in e1.input_conditions:
15                 return False
16
17             config_e1 = self.local_configuration(e1)
18             for ancestor_event in config_e1:
19                 if n2 in ancestor_event.input_conditions:
20                     return False
21             return True
22

```

Codice 4.4: Concorrenza con condizioni iniziali

Nel caso generale in cui i due eventi esistano e siano differenti, procede nel controllare la presenza di causalità tra le coppie (e_1, e_2) , o (e_2, e_1) , tramite la funzione `is_causal()`, e successivamente si controlla la presenza delle precedenti coppie all'interno del set `conflict_relation()`. Se causalità e conflitto non sono presenti, allora gli eventi sono concorrenti, e la funzione termina ritornando *True*.

```

1      if self.is_causal(e1, e2) or self.is_causal(e2, e1):
2          return False
3
4      if (e1, e2) in self.conflict_relation or (e2, e1) in self.
5         conflict_relation:
6             return False
7
8      return True

```

Codice 4.5: Concorrenza tra eventi

4.6 Ordine adeguato ed eventi cut-off

Per poter dar modo all'algoritmo di terminare la sua esecuzione con la produzione di un prefisso finito e completo di un unfolding, è necessario introdurre un ordine adeguato apposito che ne limiti la crescita, evitando di esplorare configurazioni di eventi che portano a marcature già raggiunte in sequenze di eventi considerate più piccole, identificando degli eventi cut-off. Per poter implementare ciò bisogna calcolare la configurazione locale di un evento, ovvero la sequenza causale di eventi che portano all'evento stesso, e la loro lunghezza, definire una funzione di ordine adeguato tra configurazioni sulla definizione posta in [3] e infine una funzione in grado di riconoscere gli eventi cut-off sulla base delle loro marcature.

Configurazione locale

Per poter identificare una configurazione locale è stata definita una funzione *local_configuration()*. In input viene fornito l'evento di cui si cerca la configurazione locale, mentre come risultato viene fornito l'insieme di eventi causalmente precedenti ad esso, con l'evento fornito in input incluso.

La funzione ricerca la configurazione locale tramite una visita all'indietro lungo le relazioni causali dell'evento. Come primo passo la funzione controlla l'insieme *config_cache*. Se una configurazione locale di un evento è già stata calcolata, non è necessario ripetere l'intero processo, viene quindi ritornata la configurazione locale usando l'evento come indice dell'insieme.

Successivamente viene inizializzata la pila di eventi con il nome di *stack*, insieme ad un insieme di eventi visitati *visited*. Fin quando la pila *stack* contiene elementi, si estrae il primo evento presente nella pila, si controlla che esso non faccia parte dell'insieme *visited* e, in caso contrario, lo si inserisce in questo insieme. Per ogni condizione di input dell'evento estratto dalla pila, la funzione estrae ogni evento che genera tali condizioni e li inserisce nello *stack* di ricerca. L'insieme risultante *visited* equivale alla configurazione locale ricercata. Utilizzando l'evento come indice, l'insieme *visited* viene inserito nella cache di configurazioni e successivamente ritornato al chiamante.

```
1 def local_configuration(self, event):
2     if event in self.config_cache:
3         return self.config_cache[event]
4     visited = set()
5     stack = [event]
6     while stack:
7         node = stack.pop()
8         if node in visited:
9             continue
10        visited.add(node)
11        for c in node.input_conditions:
12            if c.event is not None:
13                stack.append(c.event)
14
15        self.config_cache[event] = visited
16    return visited
17
```

Codice 4.6: La costruzione di una configurazione locale

Siccome la lunghezza della configurazione locale viene utilizzata come primo confronto per l'ordine adeguato, è stata introdotta una funzione *get_config_length()*.

Questa funzione prende un evento in input, ne calcola la configurazione locale e restituisce la lunghezza di tale configurazione. La lunghezza viene salvata in un dizionario *config_length_cache* in modo da rendere immediato il calcolo della lunghezza in caso di eventi già analizzati.

Ordine Adeguato

Come definito in [3] il confronto tra due eventi avviene tramite un confronto tra le loro configurazioni locali su tre livelli. Per poter svolgere ciò è stata implementata la funzione *compare_adequate_order()*, la quale ottiene come argomenti i due eventi su cui bisogna svolgere il confronto e fornisce come risultato un valore di verità compreso tra -1 , se il primo evento risulta avere una configurazione locale "più piccola" del secondo evento, e 1 nel caso opposto.

- Primo livello: il primo livello di confronto viene svolto dalla funzione confrontando la lunghezza delle configurazioni locali degli eventi. Se le lunghezze delle configurazioni locali risultano essere diverse, si procede al controllo della lunghezza, con il valore numerico di verità come risultato.

```

1         if len1 != len2:
2             return -1 if len1 < len2 else 1
3

```

- Secondo livello: Nel caso le configurazioni siano di dimensione uguale, la funzione procede nel confrontare l'ordine lessicografico delle configurazioni locali, in quanto due configurazioni potrebbero rappresentare sequenze causali, o occorrenze di transizioni, differenti nonostante la dimensione uguale. Per ogni evento all'interno della configurazione, si estraggono le transizioni che etichettano tali eventi, le transizioni sono poi ordinate, utilizzando la funzione di python per ottenere un identificativo unico di un oggetto, *id()*, e salvate in una tupla, una per configurazione locale. La funzione controlla, come nel confronto precedente, se gli insiemi risultano differenti e, in caso affermativo, procede al confronto delle lunghezze.

```

1         sortedLabels1 = tuple(sorted(id(e.transition) if e.
2             transition else 0 for e in config1))
3         sortedLabels2 = tuple(sorted(id(e.transition) if e.
4             transition else 0 for e in config2))
5
6         if sortedLabels1 != sortedLabels2:
7             return -1 if sortedLabels1 < sortedLabels2 else 1
8

```

- Terzo livello: Se le configurazioni coincidono sia nel primo che nel secondo livello di confronto, si passa al controllo tramite la forma normale di Foata, che consente di svolgere un confronto basato sulla struttura causale delle configurazioni, rappresentando queste ultime come sequenze ordinate su livelli di concorrenza. Per ottenere la forma normale di Foata di una configurazione è stata definita la funzione *get_foata_normal_form()*. Questa funzione ottiene come attributo una configurazione e restituisce un insieme di insiemi di eventi concorrenti.

Data una configurazione opportunamente inserita in un insieme di comodo *config_set*, la funzione svolge un ciclo finché tale insieme presenta degli

eventi, ogni iterazione rappresenta un livello della forma normale di Foata. Un evento viene considerato minimale, e inserito nella lista *min_elements*, se non presenta predecessori causali all'interno della configurazione corrente. Questo controllo viene eseguito su ogni condizione di input di ogni evento della configurazione corrente:

```

1      for e in config_set:
2          has_predecessor = False
3
4          for c in e.input_conditions:
5              if c.event is not None and c.event in
config_set:
6                  has_predecessor = True
7              break
8

```

In caso contrario l'evento *e* viene aggiunto all'insieme degli eventi minimali. Ogni livello viene inserito in una tupla di python contenente gli id() delle transizioni a cui l'evento si riferisce. Tale tupla è aggiunta nell'insieme *foata_form*, un insieme che può essere confrontato lessicograficamente in python.

```

1      slice_vector = tuple(id(e.transition) if e.transition
else 0 for e in min_elements)
2      foata_form.append(slice_vector)
3

```

Infine ogni evento minimale viene rimosso dall'insieme *config_set*, in modo da elaborare i loro successori e stratificando la configurazione. Viene quindi ritornata la forma normale di foata sotto forma di tupla di python.

```

1      for e in min_elements:
2          config_set.remove(e)
3      return tuple(foata_form)
4

```

Come nei due livelli di confronto precedenti, anche in questo caso viene restituito un valore numerico in base a quale delle due configurazione è più piccola: -1 se lo è il primo evento fornito, 1 altrimenti.

Calcolo del cut

Il cut viene utilizzato per stabilire lo stato raggiunto da una configurazione nella rete. La funzione *calculate_cut()* serve a svolgere ciò, prendendo in input una configurazione e restituendone il taglio.

Per ogni evento presente nella configurazione, la funzione conserva in due insiemi, *c_preset* e *c_postset*, tutte le condizioni di input e di output rispettivamente. Vengono raccolte tutte le condizioni iniziali non utilizzate dalla configurazione svolgendo una differenza insiemistica tra il *min* dell'unfolding e l'insieme *c_preset*. La funzione ricava il taglio svolgendo l'unione tra le condizioni non utilizzate e *c_postset* e rimuovendo l'insieme dei preset. In altre parole si ottiene il taglio $Cut(C)$, dove *C* è una configurazione, definito in [3]: $Cut(C) = (Min(O) \cup C^\bullet) \setminus C$. Il taglio *cut* rappresenta la marcatura raggiunta dalla configurazione locale di un evento *e*.

Identificazione degli eventi cut-off

La funzione *is_cutoff()* ha il compito di determinare se un evento inserito nel prefisso, e fornito in input come attributo, è un evento di tipo cut-off, o meno. Come primo passo viene calcolata la marcatura dell'evento, ovvero viene identificato il suo *cut*. Si verifica se è già presente un evento che raggiunge la stessa marcatura, controllando se la marcatura di tale evento è presente all'interno del dizionario *min_by_marking()*, un insieme contenente eventi riferiti da un indice basato sulle loro marcature. In caso affermativo, viene estratto l'evento, chiamato *min_event*, già esistente e si pone a confronto, tramite ordine adeguato, con l'evento ottenuto in input:

```
1         compare_result = self.compare_adequate_order(min_event ,
2               new_event)
```

Si ottengono quindi tre casi:

- Il risultato del confronto è < 0 , ovvero l'evento minimo già presente nel prefisso ha una configurazione locale "più piccola" dell'evento fornito come attributo in input alla funzione. Questo implica che il nuovo evento è un cut-off, comunicando all'algoritmo di unfolding principale di non espandere l'insieme delle possibili estensioni a partire da tale evento.
- Il risultato del confronto è > 0 , il nuovo evento ha una marcatura minore dell'evento minimo già presente nel prefisso, esso diventa il nuovo evento minimo raggiunto dalla sua marcatura, aggiornando l'insieme *min_by_marking* all'indice corrispondente alla marcatura dell'evento.
- Per qualsiasi altro caso, il nuovo evento non viene considerato come cut-off.

4.7 Conversione in linguaggio DOT

Per poter visualizzare una rappresentazione grafica del prefisso ottenuto è stata definita una funzione *file_to_dot()* il cui compito è quello di fornire un file in linguaggio DOT, convertibile in .png tramite graphviz, o visualizzabile online in un qualsiasi editor compatibile con i formati di graphviz. La funzione ha come attributi di input il prefisso dell'unfolding e il nome del file di output, quest'ultimo corrispondente al nome del file della rete di Petri. Come stile per rappresentare il grafo dell'unfolding è stato scelto quello fornito dai file di output di **PUNF**⁴, un software unfoldr sviluppato dall'autore di [6]: un cerchio per le condizioni e un rettangolo per gli eventi con al loro interno l'id della condizione, o dell'evento, a cui si riferiscono, oltre che al nome originale del posto e della transizione rappresentata.

Vengono sfruttati gli insiemi delle relazioni causali immediate tra condizioni ed eventi per formare le righe del file .dot. In particolare, due strutture dati di tipo set, *cond_map* e *ev_map* contengono le stringhe che rappresentano gli archi del grafo. Un insieme chiamato *elements_characteristics* contiene gli identificativi di ogni elemento dell'unfolding, usati per indicare eventi e condizioni nella visualizzazione del grafo, indicando la forma, cerchio o rettangolo, con cui l'elemento viene rappresentato.

⁴<http://homepages.cs.ncl.ac.uk/victor.khomenko/home.formal/tools/UnfoldingTools/current/>

Per ogni coppia presente in `causal_relation_pre` viene inserita in `cond_map` una f-string di python:

```
1 cond_map.append(f"c{cond.id} -> e{ev.id}")
2
```

Lo stesso, con eventi e condizioni invertite, viene fatto per l'insieme `ev_map`, sfruttando le coppie in `causal_relation_post`.

Per rappresentare graficamente gli elementi dell'unfolding, vengono inserite nell'insieme `elements_characteristics` delle stringhe contenenti l'id di ogni elemento, il nome dell'elemento e la forma di rappresentazione:

```
1 for n in unf:
2     if isinstance(n, Condition):
3         elements_characteristics.append(f'c{n.id} [label="{n.
4 place.name} (c{n.id})" shape=circle];')
5     if isinstance(n, Event):
6         elements_characteristics.append(f'e{n.id} [label="{n.
7 transition.name} (e{n.id})" shape=box];')
```

Viene quindi iniziata la scrittura su file, tramite dei cicli `for` per ogni elemento presente negli insiemi `cond_map`, `ev_map` ed `elements_characteristics`, inserendo il carattere `\n` per separare ogni stringa andando a capo, ottenendo il file in linguaggio DOT finale.

```
digraph test {
  c1 -> e1;
  e1 -> c2;
  c2 -> e2;
  c2 -> e3;
  e1 -> c3;
  c3 -> e2;
  c3 -> e3;
  e2 -> c4;
  e3 -> c5;
  c1 [label="p1 (c1)" shape=circle];
  c2 [label="p2 (c2)" shape=circle];
  c3 [label="p3 (c3)" shape=circle];
  c4 [label="p4 (c4)" shape=circle];
  c5 [label="p1 (c5)" shape=circle];
  e1 [label="t1 (e1)" shape=box];
  e2 [label="t2 (e2)" shape=box];
  e3 [label="t3 (e3)" shape=box];
}
```

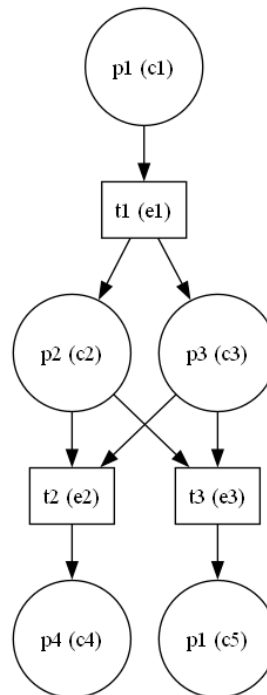


Figura 4.4: Un file in linguaggio DOT e il corrispettivo risultato grafico

4.8 Esempi di unfolding

Verranno ora presentati degli esempi di unfolding svolti con l'utilizzo dell'implementazione dell'algoritmo sviluppata durante lo stage in università.

Rete a diamante

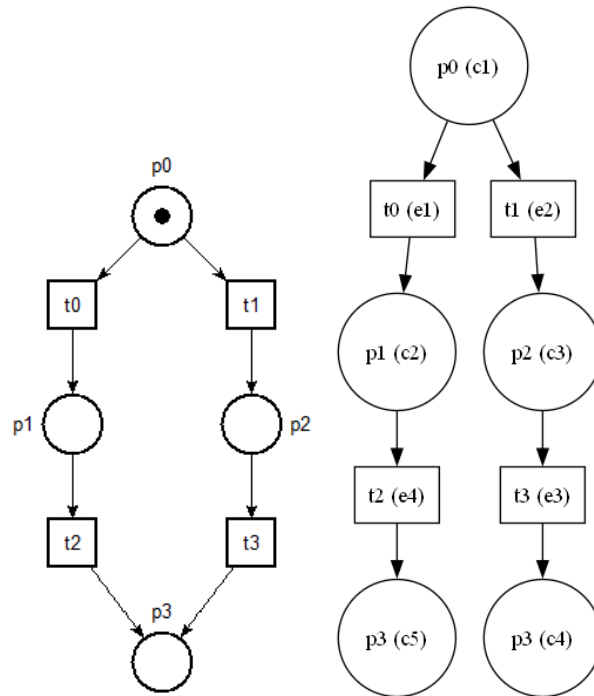


Figura 4.5: Rete a forma di diamante e relativo unfolding

Questa rete molto semplice presenta un conflitto iniziale che prosegue in due possibili esecuzioni differenti verso il posto finale p_3 . Le due possibili esecuzioni della rete sono correttamente visualizzate nel prefisso posto a destra.

Rete di mutua esclusione

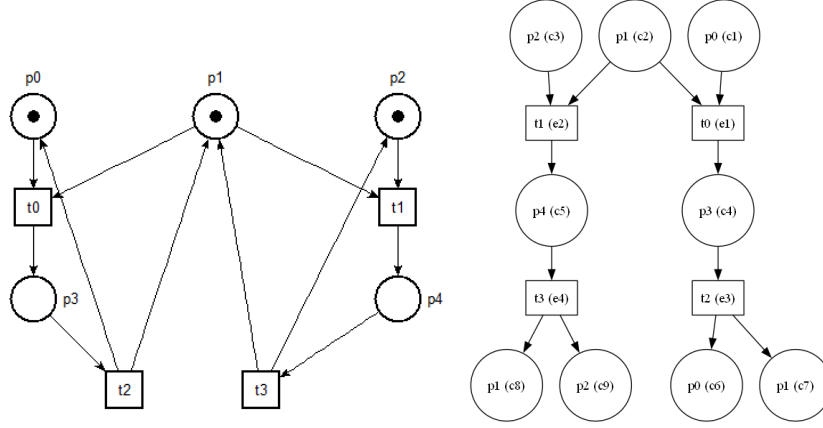


Figura 4.6: Una rete con modello di mutua esclusione ed unfolding

Una rete rappresentante una situazione di mutua esclusione tramite un conflitto iniziale. Data la configurazione di tale rete, solamente una transizione tra t_0 e t_1 può occorrere all'inizio dell'esecuzione di tale rete. Ognuno dei due possibili percorsi riporta la rete allo stato iniziale. Nel prefisso a destra, è possibile vedere i due percorsi separati, entrambi terminanti in due condizioni rappresentati le condizioni iniziali consumate, ovvero ritornanti allo stato iniziale. Grazie all'implementazione dell'ordine adeguato e degli eventi cut-off, l'esecuzione termina nonostante la natura ciclica della rete di Petri originale.

Rete con deadlock

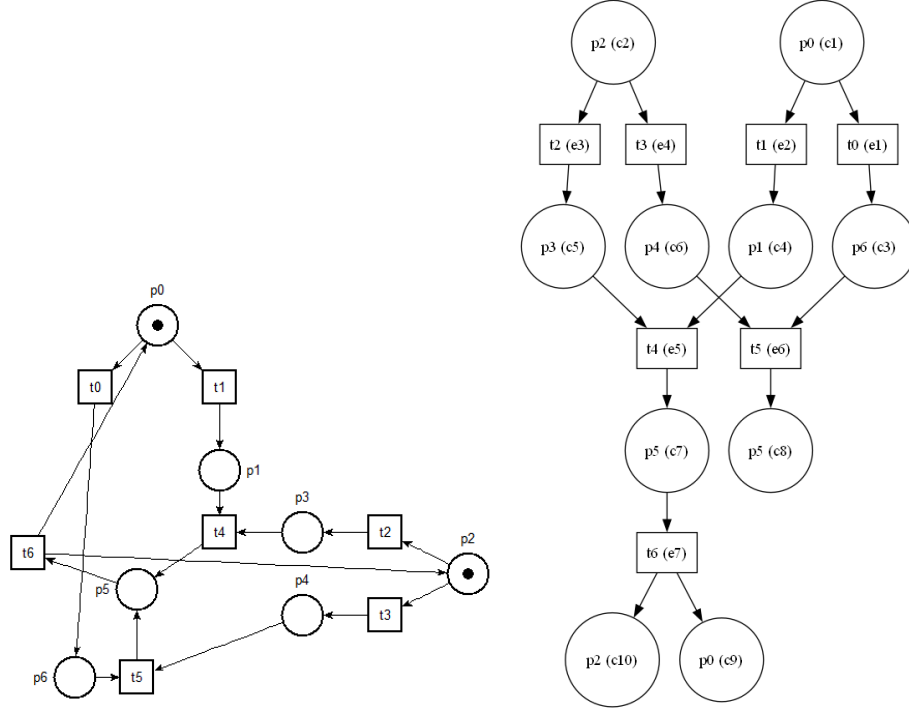


Figura 4.7: Rete con una possibile situazione di deadlock e corrispettivo prefisso

La rete in figura 4.7 presenta una situazione di esecuzione "sincronizzata". La sincronizzazione avviene nei casi in cui le transizioni eseguite inizialmente sono t_0 insieme alla transizione t_3 , oppure t_1 insieme alla transizione t_2 . Queste due possibili esecuzioni portano al posto p_5 e alla transizione t_6 che riporta la rete allo stato iniziale. Dal prefisso dell'unfolding, a destra nella figura 4.7, è possibile notare come, ad esempio, le possibili configurazioni $e_3 \rightarrow c_5$ e $e_4 \rightarrow c_3$ portino ad una situazione di stallo in cui le transizioni t_4 e t_5 non hanno le risorse necessarie per poter scattare. Si può notare come anche l'esecuzione delle transizioni t_1 e t_3 porta ad una situazione di deadlock.

Conclusioni

L'obiettivo del tirocinio, e del lavoro svolto, è stato lo studio della teoria e degli algoritmi riguardanti l'unfolding di reti di Petri e la generazione di prefissi degli stessi, con risultato lo sviluppo di un'implementazione software per generare tali prefissi.

Lo studio teorico ha seguito l'ordine con cui i concetti sono stati presentati all'interno della relazione: uno studio generale del concetto di rete di Petri e di unfolding di rete di Petri, seguito dallo studio degli algoritmi per generare un unfolding e per rendere finita la loro costruzione. L'apprendimento di strategie migliorative degli algoritmi tramite euristiche sulle transizioni, oltre che a modelli alternativi di unfolding portati alla ricerca di un determinato stato di una rete di Petri.

Il software sviluppato è in grado di produrre prefissi per unfolding di reti di Petri ordinarie, in particolare reti 1-safe, finiti e completi, coerenti con i concetti teorici di ordine adeguato e cut-off, la cui implementazione permette di eseguire una costruzione finita di un prefisso, oltre ad implementare delle limitate strutture dati di supporto impiegate per migliorare determinati processi ridondanti durante l'esecuzione dell'algoritmo. Il progetto finale presenta però numerosi limiti, dovuti alla natura di prototipo di implementazione del programma. Il programma sviluppato non tiene conto della complessità computazionale e della memoria utilizzata, dando priorità ad uno sviluppo corretto e funzionante dell'algoritmo, rispetto alle sue prestazioni. Non sono presenti controlli per verificare la correttezza strutturale della rete di Petri come in caso di reti non *bounded*. Inoltre il programma implementa la generazione di prefisso classica, senza possibilità di utilizzare tecniche di unfolding alternative, come l'impiego di ordini adeguati differenti o la tecnica del directed unfolding.

Come possibili sviluppi futuri, oltre all'implementazione di controlli sulla correttezza della rete di Petri, si potrebbero implementare miglioramenti al codice stesso dell'implementazione, oltre che ad ottimizzare le strutture dati utilizzate: un esempio è diminuire il numero di strutture dati utilizzate per rappresentare le relazioni causali, da due ad una sola. L'introduzione di metodi di unfolding alternativi, come il directed unfolding esposto nel capitolo 3 o il *goal-driven unfolding* [8], non trattato in questo elaborato. Inoltre l'introduzione di tecniche di unfolding per differenti classi di reti di Petri, come ad esempio le reti di Petri colorate o le reti di Petri temporizzate, potrebbe risultare uno sviluppo interessante, espandendo gli ambiti di utilizzo del software. Come ulteriore sviluppo, metodi di analisi automatizzati dell'unfolding, come l'identificazione automatica di stati deadlock o di raggiungibilità degli stati, risultano essere la naturale evoluzione del programma, per poter offrire degli utilizzi concreti di tale implementazione.

Inoltre, grazie alla rappresentazione in classi della rete di Petri, è anche possibile implementare un simulatore di esecuzione della rete stessa.

In conclusione, da un punto di vista personale, il lavoro svolto durante lo sviluppo di un algoritmo per unfolding, e lo studio teorico riguardante le reti di Petri, ha portato ad una migliore comprensione della modellazione di sistemi concorrenti. Lo sviluppo stesso dell'implementazione software ha portato ad una più profonda conoscenza dei processi fondamentali dello sviluppo, quali la scelta delle strutture dati adeguate per la rappresentazione di insiemi, oltre che ad una più chiara comprensione di come un concetto teorico possa diventare realtà tramite un'implementazione concreta.

Bibliografia

- [1] Luca Bernardinello and Fiorella De Cindio. A survey of basic net models and modular net classes. In *Advances in Petri Nets 1992*, pages 304–351. Springer, 2005.
- [2] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 2002.
- [3] Javier Esparza, Stefan Römer, and Walter Vogler. An improvement of McMillan’s unfolding algorithm. *Formal Methods in System Design*, 20(3):285–310, 2002.
- [4] Blai Bonet, Patrik Haslum, Sarah Hickmott, and Sylvie Thiébaux. Directed unfolding of Petri nets. In *Transactions on Petri Nets and Other Models of Concurrency I*, pages 172–198. Springer, 2008.
- [5] Victor Khomenko and Maciej Koutny. An efficient algorithm for unfolding Petri nets. Technical report, Technical Report CS-TR-726, Department of Computing Science, University of Newcastle, 2001.
- [6] Victor Khomenko. *Model checking based on prefixes of Petri net un-foldings*. PhD thesis, PhD thesis, University of Newcastle, 2003.
- [7] Joost Engelfriet. Branching processes of Petri nets. *Acta Informatica*, 28(6):575–591, 1991.
- [8] Thomas Chatain and Loïc Paulevé. Goal-driven unfolding of Petri nets. *arXiv preprint arXiv:1611.01296*, 2016.
- [9] Kenneth L. McMillan and David K. Probst. A technique of state space search based on unfolding. *Formal methods in system design*, 6(1):45–65, 1995.
- [10] César Rodríguez and Stefan Schwoon. An improved construction of Petri net unfoldings. In Christine Choppy and Jun Sun, editors, *1st French Singaporean Workshop on Formal Methods and Applications, FSFMA 2013, July 15-16, 2013, Singapore*, volume 31 of *OASICS*, pages 47–52. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2013.
- [11] Kenneth L. McMillan. Using unfoldings to avoid the state explosion problem in the verification of asynchronous circuits. In Gregor von Bochmann and David K. Probst, editors, *Computer Aided Verification, Fourth International Workshop, CAV ’92, Montreal, Canada, June 29 - July 1,*

- 1992, *Proceedings*, volume 663 of *Lecture Notes in Computer Science*, pages 164–177. Springer, 1992.
- [12] Antti Valmari. The state explosion problem. In *Advanced Course on Petri Nets*, pages 429–528. Springer, 1996.
 - [13] Javier Esparza and Claus Schröter. Unfolding based algorithms for the reachability problem. *Fundam. Informaticae*, 47(3-4):231–245, 2001.
 - [14] Javier Esparza and Keijo Heljanko. *Unfoldings - A Partial-Order Approach to Model Checking*. Monographs in Theoretical Computer Science. An EATCS Series. Springer, 2008.
 - [15] Stephan Melzer and Stefan Römer. Deadlock checking using net unfoldings. In Orna Grumberg, editor, *Computer Aided Verification, 9th International Conference, CAV '97, Haifa, Israel, June 22-25, 1997, Proceedings*, volume 1254 of *Lecture Notes in Computer Science*, pages 352–363. Springer, 1997.